

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

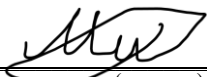
Факультет безопасности информационных технологий

Дисциплина:
«Компьютерные сети»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
«Анализ трафика компьютерных сетей утилитой Wireshark»

Выполнили:

Матвеев Илья Александрович, студент группы N3351



(подпись)

Проверил:

Будько Михаил Юрьевич, доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Сканирование и анализ сетевого трафика	4
1.1 Анализ трафика утилиты ping	4
1.2 Анализ утилиты tracerout	8
1.3 Анализ HTTP-трафика	13
1.4 Анализ DNS-трафика	15
1.5 Анализ протокола ARP	18
1.6 Анализ dns-пакетов с помощью утилиты nslookup	22
1.7 Анализ FTP-трафика	25
1.8 Анализ DHCP-трафика	28
1.9 Анализ трафика мессенджера Telegram	33
Заключение	36

ВВЕДЕНИЕ

Цель работы – изучить структуру протокольных блоков данных, анализируя реальный трафик на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark

Для достижения поставленной цели необходимо решить следующие задачи:

- Локально установить утилиту Wireshark;
- Проанализировать каждую консольную утилиту и то, как отправленные с помощью утилиты пакеты отображаются в Wireshark;
- Подготовить и оформить отчет по лабораторной работе.

1 СКАНИРОВАНИЕ И АНАЛИЗ СЕТЕВОГО ТРАФИКА

В данном разделе представлено выполнение всех заданий, представленных во введении.

1.1 Анализ трафика утилиты ping

Главная цель - исследовать особенности передачи ICMP Echo Request/Echo Reply, выявить фрагментацию IP-пакетов при различных размерах данных, проанализировать механизм Path MTU Discovery, изменение TTL и структуру ICMP-данных.

Использовался сайт mail.ru, содержащий буквы из ФИО – Матвеев Илья Александрович (mia).

В ходе эксперимента были применены команды, представленные в листинге 1.

Листинг 1 – Примененные команды

```
ping mail.ru
ping -s 100 mail.ru
ping -s 500 mail.ru
ping -s 1000 mail.ru
ping -s 2000 mail.ru
ping -s 3000 mail.ru
ping -s 4000 mail.ru
ping -s 6000 mail.ru
ping -s 8000 mail.ru
ping -s 10000 mail.ru

ping -s 2000 -M do mail.ru
ping -s 2000 -M dont mail.ru

ping -s 2000 -t 10 mail.ru
ping -s 2000 -t 60 mail.ru
```

Для анализа трафика в данном разделе и в последующих используется утилита Wireshark. Фильтр, используемый при анализе: icmp. Wireshark запускался до выполнения ping и непрерывно собирал трафик.

6835.. 1074.1677721.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=1/256, ttl=64	(reply in 683568)
6835.. 1074.1972982.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=1/256, ttl=56	(request in 683559)
6841.. 1075.1691632.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=2/512, ttl=64	(reply in 684184)
6841.. 1075.1852791.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=2/512, ttl=56	(request in 684166)
6849.. 1076.1697814.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=3/768, ttl=64	(reply in 685095)
6850.. 1076.2201233.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=3/768, ttl=56	(request in 684952)
6856.. 1077.1715443.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=4/1024, ttl=64	(reply in 685698)
6856.. 1077.1816917.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=4/1024, ttl=56	(request in 685677)
6864.. 1078.1734846.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=5/1280, ttl=64	(reply in 686428)
6864.. 1078.2030598.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=5/1280, ttl=56	(request in 686402)
6869.. 1079.1755078.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=6/1536, ttl=64	(reply in 686951)
6869.. 1079.1935184.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=6/1536, ttl=56	(request in 686939)
6875.. 1080.1767226.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=7/1792, ttl=64	(reply in 687621)
6876.. 1080.2633829.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=7/1792, ttl=56	(request in 687589)
6884.. 1081.1777533.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=8/2048, ttl=64	(reply in 688506)
6885.. 1081.2911675.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=8/2048, ttl=56	(request in 688409)
6891.. 1082.1706866.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=9/2304, ttl=64	(reply in 689212)
6892.. 1082.2668156.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=9/2304, ttl=56	(request in 689188)
6899.. 1083.1797790.. 172.28.126.227	90.156.232.4	ICMP	1162 Echo (ping) request	id=0x1f35, seq=10/2560, ttl=64	(reply in 689918)
6900.. 1083.2653866.. 90.156.232.4	172.28.126.227	ICMP	1162 Echo (ping) reply	id=0x1f35, seq=10/2560, ttl=56	(request in 689989)

Рисунок 1 – Список ICMP-пакетов в Wireshark для большого размера

На рисунке 1 видно, что один Echo Request разбивается на серию IP-фрагментов, что определяется по одинаковому полю Identification и различным смещениям.

7641.. 1283.6672183.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=1/256, ttl=10 (reply in 764206)
7642.. 1283.6921614.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=1/256, ttl=56 (request in 764188)
7649.. 1284.6676928.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=2/512, ttl=10 (no response found)
7656.. 1285.6887667.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=3/768, ttl=10 (no response found)
7663.. 1286.7127221.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=4/1024, ttl=10 (reply in 766424)
7664.. 1286.7658993.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=4/1024, ttl=56 (request in 766378)
7669.. 1287.7130576.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=5/1280, ttl=10 (no response found)
7675.. 1288.7630710.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=5/1536, ttl=10 (reply in 767800)
7676.. 1288.7873737.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=6/1536, ttl=56 (request in 767583)
7683.. 1289.7616762.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=7/1792, ttl=10 (reply in 768494)
7684.. 1289.8677299.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=7/1792, ttl=56 (request in 768387)
7691.. 1210.7656822.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=8/2048, ttl=10 (reply in 769167)
7691.. 1210.8348898.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=8/2048, ttl=56 (request in 769126)
7698.. 1211.7635709.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=9/2304, ttl=10 (reply in 769899)
7698.. 1211.7868826.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=9/2304, ttl=56 (request in 769874)
7706.. 1212.7646428.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=10/2560, ttl=10 (reply in 770658)
7706.. 1212.8039524.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=10/2560, ttl=56 (request in 770633)
7713.. 1213.7662068.. 172.28.126.227	89.221.239.1	ICMP	562 Echo (ping) request	id=8x207a, seq=11/2816, ttl=10 (reply in 771421)
7714.. 1213.8281236.. 89.221.239.1	172.28.126.227	ICMP	562 Echo (ping) reply	id=8x207a, seq=11/2816, ttl=56 (request in 771361)

Рисунок 2 – Пакет с изменённым TTL

TTL меняется параметром -t. В Wireshark видно уменьшенное значение TTL в поле Time-To-Live.

В таблице 1 представлен результат анализа.
Таблица 1 – Результат анализа

Размер данных ICMP	IP размер (payload + 28)	Фрагментация	Кол-во фрагментов	Наблюдаемое смещение (пример из Wireshark)
100	128	нет	1	0
500	528	нет	1	0
1000	1028	нет	1	0
2000	2028	да	2	~1500
3000	3028	да	2	~1500
4000	4028	да	3	~3000
6000	6028	да	4	~4500
8000	8028	да	6	~6000
10000	10028	да	7	~7500

Анализ таблицы показывает, что при малых размерах ICMP-даты (до 1000 байт) фрагментация не возникает: каждый запрос передается одним IP-пакетом, что указывает на достаточный размер MTU по умолчанию для прохождения таких сообщений.

При увеличении нагрузки свыше 1500 байт (с учётом заголовков IP и ICMP) наблюдается фрагментация — один Echo Request разбивается на несколько IP-пакетов. Это

подтверждается наличием одинакового поля Identification и возрастающими значениями Fragment Offset, фиксируемыми в Wireshark.

Число фрагментов возрастает ступенчато, пропорционально увеличению ICMP-поля данных. Пороговое значение соответствует MTU ~1500 байт: полезная нагрузка фрагмента составляет около 1480 байт, поэтому каждый последующий фрагмент содержит аналогичный объём данных и увеличенный offset.

Для 2000–3000 байт наблюдается деление на два фрагмента, для 4000 байт — уже три, а для 6000 и 8000 — четыре и шесть соответственно. Для 10000 байт трафик разбился на семь фрагментов. Такая зависимость демонстрирует прямую связь между MTU и механизмом деления IP-пакетов на части.

Также из таблицы видно, что с ростом размера увеличивается вероятность потерь — некоторые фрагменты отсутствуют, что фиксируется как «no reply» в ping и отсутствие соответствующего Echo Reply в Wireshark. Это объясняется тем, что потеря хотя бы одного фрагмента приводит к недоставке всего Echo Request/Reply и повышает фактический packet loss.

Далее, на рисунке 3 представлена зависимость количества фрагментов от размера пакета.

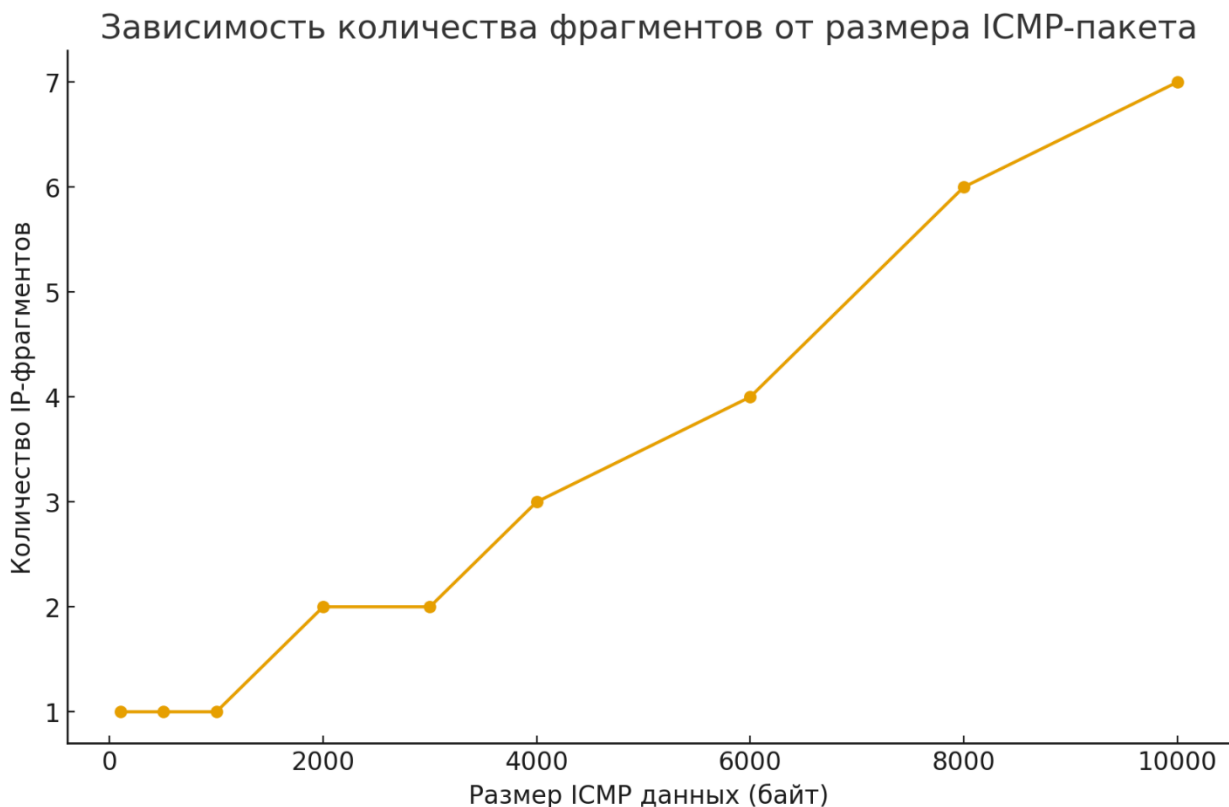


Рисунок 3 – График

График демонстрирует характерную ступенчатую зависимость: при размере порядка 100–1000 байт передача осуществляется в одном IP-пакете, затем при превышении порога MTU величина числа фрагментов резко возрастает.

Рост количества фрагментов не является линейным — на графике видны скачкообразные изменения: пересечение очередного кратного размера MTU приводит к добавлению ещё одного фрагмента. Таким образом, функция количества фрагментов является кусочно-линейной с целочисленными скачками.

Это наглядно иллюстрирует внутренний принцип функционирования сетевого уровня: IP не может передать пакет, превышающий MTU, и вынужден разделять его на части фиксированной длины.

Наибольший наклон графика наблюдается в диапазоне 6000–10000 байт — в соответствии с увеличением числа фрагментов при той же емкости полезных данных. Это подтверждает, что экспоненциальный рост числа фрагментов оказывает негативное влияние на стабильность передачи (повышенный packet loss из таблицы).

В совокупности анализ графика и таблицы показывает, что оптимальный размер ICMP-данных для стабильной передачи в условиях интернета находится в пределах MTU, а превышение его ведёт к фрагментации, усложнению маршрутизации и повышению вероятности потерь.

Анализ и ответы на вопросы.

В ходе анализа ICMP-трафика выявлено, что при увеличении размера сообщений ping происходит фрагментация IP-пакетов. Wireshark позволяет наблюдать структуру фрагментов: одинаковое значение Identification, смещения Fragment Offset и флаг MF. Механизм Path MTU Discovery корректно работает — при запрете фрагментации узел сообщает ошибку MTU. TTL изменяется параметром -t, что отражается в IP-заголовке.

Поле данных содержит служебную нагрузку, увеличивающуюся вместе с параметром -s.

Ответы на вопросы:

- **Есть ли фрагментация пакетов ping с увеличенным размером?** Да. В Wireshark видно, что один Echo Request передаётся несколькими IP-фрагментами с одинаковым полем Identification, различными Fragment Offset и флагом MF (More Fragments);
- **Как определить последний фрагмент?** Последний фрагмент имеет MF=0 (More Fragments cleared) и максимальный Fragment Offset;

- **Сколько фрагментов у разных размеров?** Количество фрагментов растёт с увеличением размера данных (см. таблицу и график). Например, 6000 байт разбиваются на 4 фрагмента, 10000 — на 7;
- **Как изменить TTL и где увидеть?** TTL изменяется параметром `-t` в Linux, а его значение отображается в Wireshark в поле Time To Live заголовка IP;
- **Что содержится в поле данных?** В поле Data ICMP содержится полезная нагрузка фиксированного или изменяемого размера, состоящая из служебных байтов/шаблонов. Wireshark показывает её в шестнадцатеричном и символьном виде.

1.2 Анализ утилиты tracerout

Для исследования механизма определения маршрута до удалённого узла была использована утилита `traceroute` в ОС Linux. Одновременно с запуском утилиты выполнялся захват трафика в программе **Wireshark** с фильтром по протоколу ICMP, что позволяет отследить служебные ответы маршрутизаторов на пути следования пакетов.

Целевым узлом выбран сервер **mail.ru** (адрес 89.221.239.1). Команда запуска трассировки представлена в листинге 2.

Листинг 2 – Команда запуска утилиты

```
traceroute mail.ru
```

Фактический вывод утилиты представлен в листинге 3.

Листинг 3 – Вывод утилиты

```
traceroute to mail.ru (89.221.239.1), 30 hops max, 60 byte packets
 1 _gateway (172.28.64.1)  2.237 ms  2.214 ms  3.498 ms
 2 * 77.234.216.48 (77.234.216.48)  3.038 ms  3.028 ms
 3 ae8-10209.RT.KM.SPB.retn.ru (139.45.232.230)  3.025 ms *  4.515 ms
 4 ae3-4.RT.M9.MSK.retn.ru (139.45.243.4)  16.233 ms  21.377 ms  16.214 ms
 5 * * *
 6 * * *
 7 * * *
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
```

```

22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *

```

По данным вывода видно, что первые четыре узла по пути до mail.ru отвечают на диагностические запросы, а начиная с 5-го перехода ответы не приходят (во всех трёх попытках выводятся «*»).

No.	Time	Source	Destination	Protocol	Length	Info
95495	146.321515949	172.28.64.1	172.28.126.227	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
95496	146.321516000	172.28.64.1	172.28.126.227	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
95498	146.322373283	77.234.216.48	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95499	146.322373299	77.234.216.48	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95500	146.322380378	139.45.232.230	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95501	146.322811660	172.28.64.1	172.28.126.227	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
95502	146.323891101	139.45.232.230	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95520	146.335619548	139.45.243.4	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95521	146.335619611	139.45.243.4	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
95529	146.340773036	139.45.243.4	172.28.126.227	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
1068...	165.567396115	172.28.126.227	172.28.64.1	ICMP	370	Destination unreachable (Port unreachable)
1268...	204.110109899	172.28.126.227	172.28.64.1	ICMP	370	Destination unreachable (Port unreachable)

Рисунок 4 – список ICMP-пакетов

На рисунке 4 показан список перехваченных ICMP-пакетов. В столбцах **Source** и **Destination** видно, что:

- в качестве получателя выступает локальный хост с адресом 172.28.126.227;
- источниками ICMP-сообщений являются последовательно:
 - 172.28.64.1 — локальный шлюз / первый маршрутизатор;
 - 77.234.216.48;
 - 139.45.232.230 (узел провайдера RETN в Санкт-Петербурге);
 - 139.45.243.4 (узел RETN в Москве).

Тип ICMP-сообщений для этих узлов — **Time-to-live exceeded (Type 11, Code 0)**. Это означает, что к каждому промежуточному маршрутизатору приходит пакет с ограниченным значением TTL, TTL уменьшается до нуля, и узел отправляет обратно в сторону источника ICMP-сообщение о превышении времени жизни.

В конце списка видны пакеты **Destination unreachable (Port unreachable)** от узла, ближайшего к целевому серверу. Эти ICMP-сообщения используются traceroute как признак достижения конечного узла (или его пограничного маршрутизатора), когда датаграмма дошла до адресата, но UDP-порт в целевой системе не открыт.

Таким образом, рисунок с таблицей в Wireshark подтверждает, что трассировка строится именно на основе ICMP-ответов **Time Exceeded** от промежуточных маршрутизаторов и **Destination Unreachable** от конечной точки.

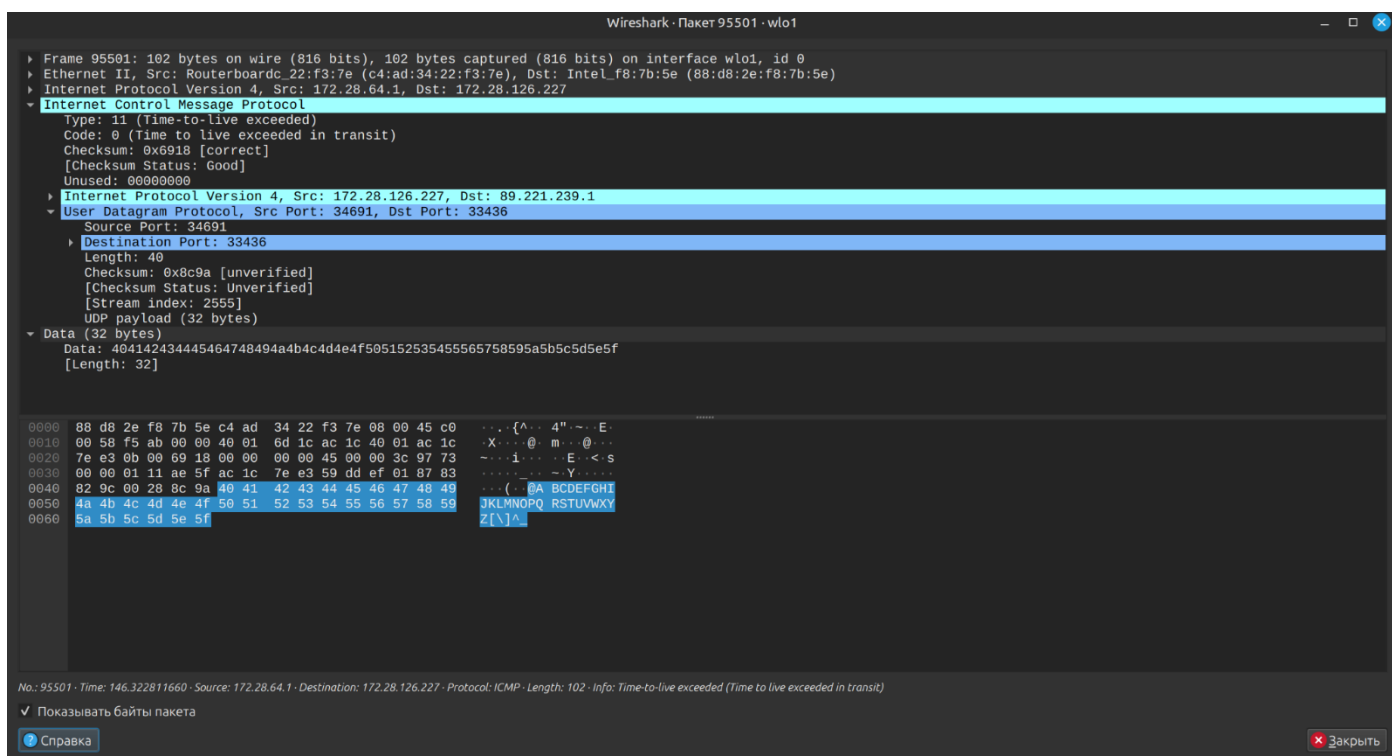


Рисунок 5 – структура ICMP Time Exceeded с вложенным IP+UDP

На рисунке 5 показана детальная структура ICMP-пакета типа *Time-to-live exceeded in transit*:

В верхней части дерева протоколов:

- **Internet Protocol Version 4, Src: 172.28.64.1, Dst: 172.28.126.227**
— внешний IP-заголовок ICMP-ответа от первого маршрутизатора.

- **Internet Control Message Protocol**

- Type: 11 (Time-to-live exceeded)
- Code: 0 (Time to live exceeded in transit)
— служебное сообщение о том, что TTL входящего пакета обнулится.

Далее во вложенной части ICMP-сообщения:

- **Internet Protocol Version 4, Src: 172.28.126.227, Dst: 89.221.239.1**
— фрагмент *оригинального* IP-пакета, отправленного утилитой traceroute.
- **User Datagram Protocol, Src Port: 34691, Dst Port: 33436**
— UDP-датаграмма, сгенерированная traceroute.
- В поле **Data (32 bytes)** виден полезный нагрузочный блок, содержащий последовательность байтов, которые служат для идентификации запроса.

Из этого рисунка хорошо видно, как работает протокол: маршрутизатор возвращает не только ICMP-заголовок, но и фрагмент исходного IP+UDP-пакета. По содержимому

вложенного заголовка утилита на стороне отправителя понимает, к какому именно запросу относится полученный ICMP-ответ.

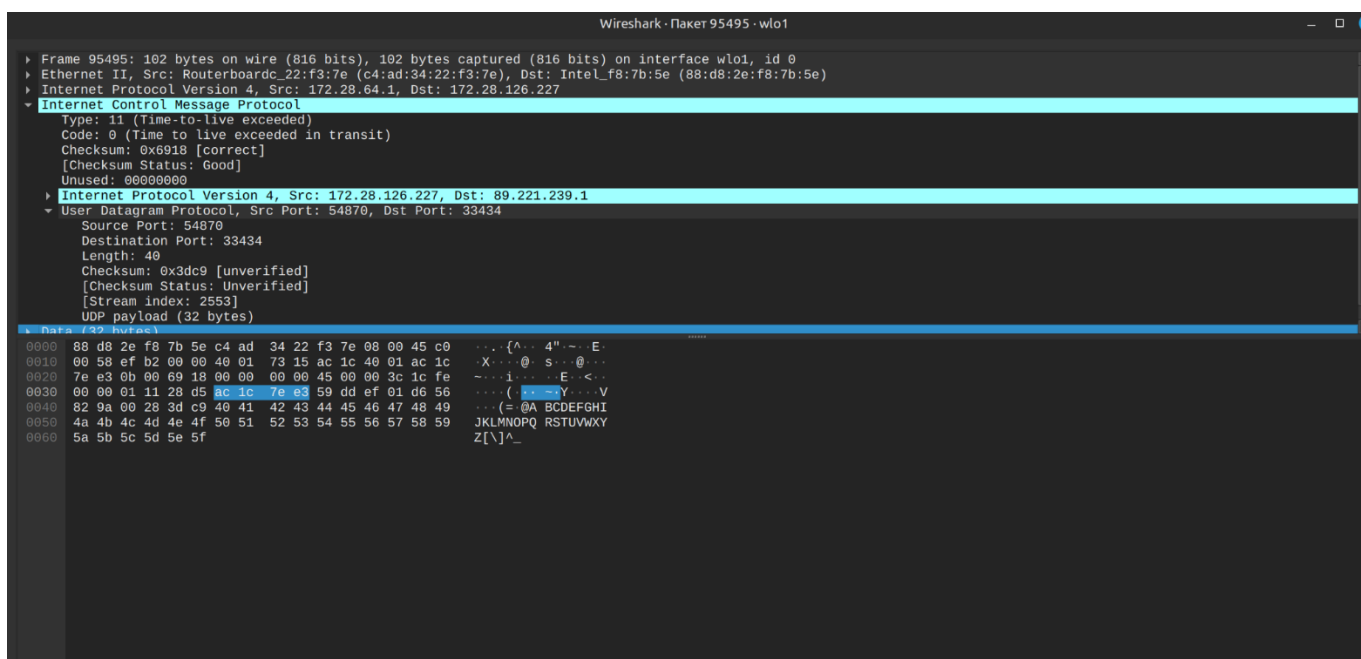


Рисунок 6 – ICMP Time Exceeded с отличающимися UDP-портами

На рисунке 6 отображён ещё один ICMP-пакет **Time-to-live exceeded**. Его структура аналогична предыдущему случаю:

- внешний IP-заголовок: **Src: 172.28.64.1, Dst: 172.28.126.227;**
- ICMP Type 11 / Code 0;
- во вложенном IP-заголовке снова исходящий пакет от 172.28.126.227 к 89.221.239.1,
- но в блоке UDP указаны уже другие порты: **Src Port: 54870, Dst Port: 33434.**

Этот рисунок подчёркивает ещё одну особенность работы traceroute: утилита использует **последовательность UDP-датаграмм с разными номерами портов**, что позволяет ей однозначно сопоставлять входящие ICMP-ответы с исходящими запросами. На уровне Wireshark это хорошо видно по различающимся значениям Source/Destination Port во вложенном UDP-заголовке.

На основе вывода traceroute и анализа ICMP-пакетов в Wireshark можно составить сводную таблицу маршрута – таблицу 2 (значения задержек взяты из вывода утилиты).

Таблица 2 – Таблица результатов трассировки

№ перехода	IP-адрес узла / имя	Средняя задержка, мс	Тип ICMP-ответа
1	172.28.64.1 (_gateway)	≈ 2.6	Time Exceeded
2	77.234.216.48	≈ 3.0	Time Exceeded
3	139.45.232.230 (ae8-10209.RT.KM.SPB.retn.ru)	≈ 3.8	Time Exceeded
4	139.45.243.4 (ae3-4.RT.M9.MSK.retn.ru)	≈ 17.9	Time Exceeded
5–30	*	нет ответа	нет ICMP-ответов

Анализ таблицы и выводы.

1. Количество доступных хопов. По таблице видно, что утилите удаётся получить ICMP-ответы только от первых четырёх маршрутизаторов. Это достаточно типичная ситуация: на магистральных или пограничных узлах провайдеры часто ограничивают генерацию ICMP-сообщений по соображениям безопасности и снижения нагрузки;
2. Рост задержек по мере удаления. Средняя задержка до локального шлюза составляет порядка 2–3 мс, до узлов провайдера — около 3–4 мс, а при переходе на московский узел время увеличивается до ~17–21 мс. Это отражает реальное географическое удаление и дополнительную длину физического канала между городами;
3. Механизм работы traceroute. Наблюдение ICMP-сообщений Time Exceeded с вложенными IP+UDP-заголовками в Wireshark подтверждает, что traceroute реализует трассировку за счёт поэтапного увеличения значения TTL в исходящих пакетах, получения от каждого промежуточного маршрутизатора ICMP-ответа при обнулении TTL, анализа вложенного заголовка исходного IP+UDP-пакета.
4. Отсутствие ответов не означает недоступности сервера. Начиная с 5-го перехода, в выводе traceroute отображаются «звёздочки», однако это не значит, что сервер mail.ru недоступен. Это лишь говорит о том, что на

дальнейших участках пути ICMP-пакеты фильтруются, а диагностические ответы не отправляются в сторону клиента. При этом обычные пользовательские протоколы (HTTP, HTTPS и т.п.) могут успешно работать поверх этого маршрута;

5. Совместный анализ traceroute и Wireshark. Совмещение текстового вывода traceroute с анализом ICMP-пакетов в Wireshark позволяет не только увидеть список маршрутизаторов, но и детально изучить формат служебных ICMP-сообщений и вложенных в них IP/UDP-заголовков, что важно для понимания реальной работы протокольного стека.

1.3 Анализ HTTP-трафика

Для исследования механизма работы протокола HTTP использовался веб-браузер Chrome и анализатор сетевого трафика Wireshark. Фильтр трафика устанавливался как: http

Это позволило выделить только HTTP-пакеты и рассмотреть структуру запросов и ответов. Целевым ресурсом выбран веб-сайт, доступный по протоколу HTTP без шифрования, что позволяет увидеть содержимое запросов и заголовков.

На рисунке 7 показан HTTP-пакет с запросом клиента.

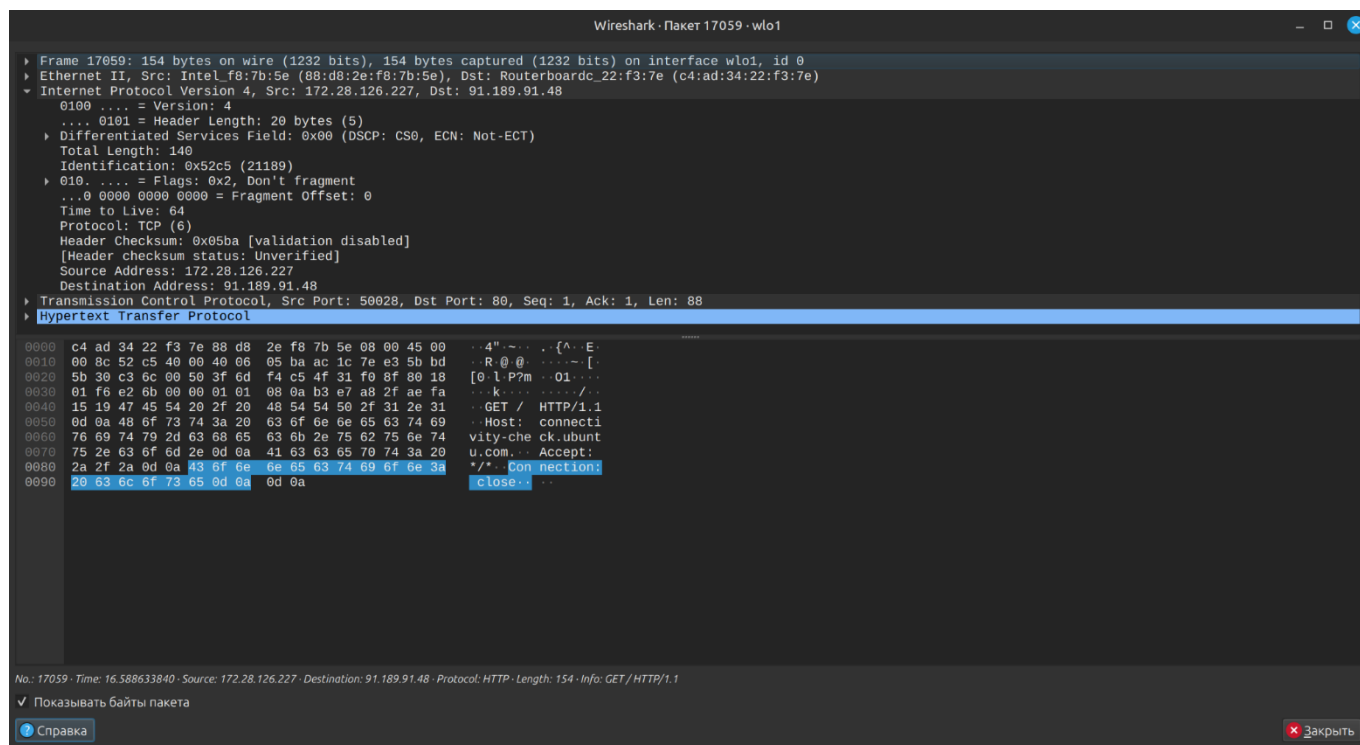


Рисунок 7 – HTTP-пакет с запросом клиента

- тип запроса — GET
- строка запроса: GET / HTTP/1.1
- порт назначения — 80
- сервер — 91.189.91.48

В раскрытой структуре пакета можно наблюдать основные заголовки, формируемые браузером:

- Host — имя целевого узла
- User-Agent — идентификация браузера
- Асерт — перечень поддерживаемых форматов
- Connection: close — запрос закрыть соединение после ответа

Скрин содержит поле Hypertext Transfer Protocol, подтверждающее отправку запроса на уровень приложения. Этот кадр иллюстрирует базовый механизм HTTP — клиент инициирует запрос, указав метод (GET), URI и параметры взаимодействия

После обновления страницы браузером был зафиксирован новый HTTP-ответ с кодом:

HTTP/1.1 304 Not Modified

Этот пакет изображён на рисунке 8.

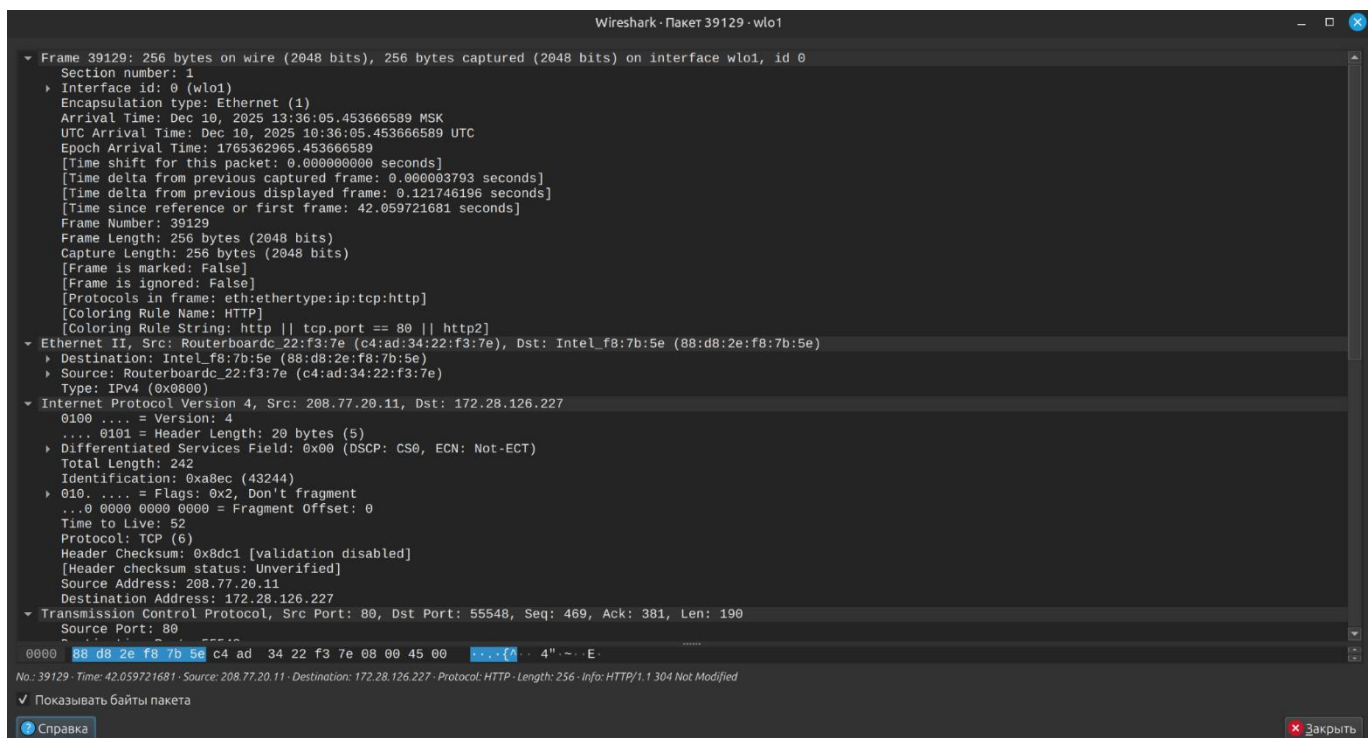


Рисунок 8 – Новый пакет

Статус **304** означает, что клиент ранее уже получал актуальную версию ресурса, и сервер сообщает, что контент не изменился — поэтому повторная передача данных не требуется.

Это является результатом механизма кэширования: клиент отправил условный запрос (с заголовками If-Modified-Since / ETag), сервер сверил дату/версию и вернул экономичный ответ **без тела страницы**.

Выводы по анализу.

Анализ первого запроса показывает классическую схему HTTP:

клиент отправляет запрос GET на сервер, который отдает ответ с кодом состояния.

Отсутствие содержимого при 204 No Content демонстрирует, что HTTP позволяет серверу корректно обработать запрос без возврата HTML-страницы.

Повторное обращение привело к кэшированному обмену — статус 304 Not Modified означает, что данные у клиента уже актуальны, и повторная передача не требуется. Это снижает сетевую нагрузку и ускоряет работу приложения.

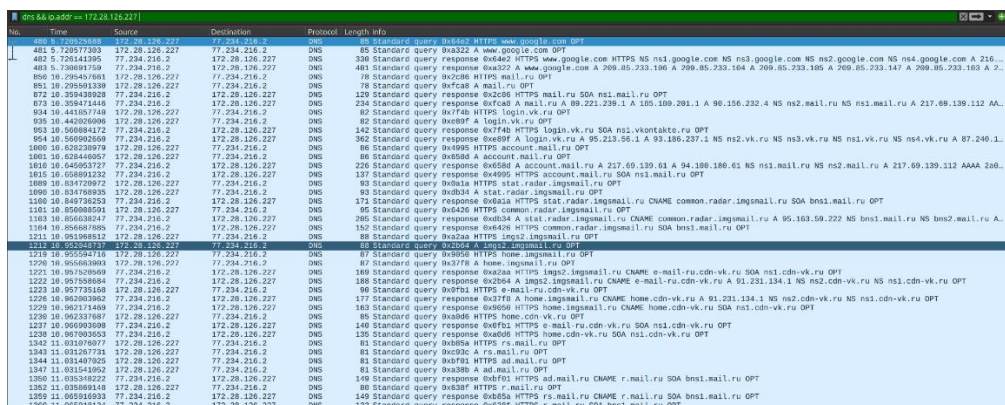
Таким образом, захваченный трафик позволяет увидеть структуру HTTP-запросов и работу механизма кэширования на прикладном уровне

1.4 Анализ DNS-трафика

Для исследования работы системы доменных имён использовалась программа **Wireshark**, установлен фильтр:

```
dns && ip.addr == 172.28.126.227
```

Это позволило выделить только DNS-пакеты, соответствующие обращениям моего компьютера. Для генерации DNS-трафика в браузере была открыта страница mail.ru, что инициировало запросы на разрешение доменных имён.



No.	Time	Source	Destination	Protocol	Length	Info
480	0.72533488	172.28.126.227	172.234.216.2	DNS	88	Standard query 0x6322 A www.google.com OPT
481	0.726177003	172.28.126.227	172.234.216.2	DNS	88	Standard query 0x6322 A www.google.com OPT
482	0.726141395	172.234.216.2	172.28.126.227	DNS	130	Standard query response 0x6322 A www.google.com HTTPS NS ns1.google.com NS ns3.google.com NS ns2.google.com NS ns4.google.com A 216...
483	0.726097158	172.234.216.2	172.28.126.227	DNS	461	Standard query response 0x6322 A www.google.com A 209.85.233.106 A 209.85.233.104 A 209.85.233.105 A 209.85.233.103 A 2...
484	0.7260451681	172.28.126.227	172.234.216.2	DNS	78	Standard query 0x6322 A mail.ru OPT
485	0.7260501338	172.28.126.227	172.234.216.2	DNS	78	Standard query 0x6322 A mail.ru OPT
486	0.7260450928	172.28.126.227	172.234.216.2	DNS	120	Standard query response 0x6322 A mail.ru SOA ns1.mail.ru OPT
487	0.7260451448	172.234.216.2	172.28.126.227	DNS	234	Standard query response 0x6322 A mail.ru A 88.221.239.1 A 185.109.291.1 A 96.156.232.4 NS ns2.mail.ru NS ns1.mail.ru A 217.69.139.132 AA...
488	0.7260451748	172.28.126.227	172.234.216.2	DNS	82	Standard query 0x6322 A login.vk.ru OPT
489	0.7260450988	172.28.126.227	172.234.216.2	DNS	142	Standard query response 0x6322 A login.vk.ru SOA ns1.vkontakte.ru OPT
490	0.7260451702	172.234.216.2	172.28.126.227	DNS	362	Standard query response 0x6322 A login.vk.ru A 95.213.55.1 A 93.186.237.1 NS ns2.vk.ru NS ns3.vk.ru NS ns4.vk.ru A 87.240.1...
491	0.7260450988	172.28.126.227	172.234.216.2	DNS	86	Standard query 0x6322 A account.mail.ru OPT
492	0.7260450988	172.28.126.227	172.234.216.2	DNS	86	Standard query 0x6322 A account.mail.ru OPT
493	0.7260450988	172.28.126.227	172.234.216.2	DNS	226	Standard query response 0x6322 A account.mail.ru A 217.69.139.61 A 94.180.180.61 NS ns1.mail.ru NS ns2.mail.ru A 217.69.139.132 AAAA 2a6...
494	0.7260450988	172.28.126.227	172.234.216.2	DNS	137	Standard query response 0x6322 A account.mail.ru SOA ns1.mail.ru OPT
495	0.7260450988	172.28.126.227	172.234.216.2	DNS	83	Standard query 0x6322 A stat.radar.ingmail.ru OPT
496	0.7260450988	172.28.126.227	172.234.216.2	DNS	171	Standard query response 0x6322 A stat.radar.ingmail.ru CNAME common.radar.ingmail.ru SOA bns1.mail.ru OPT
497	0.7260450988	172.28.126.227	172.234.216.2	DNS	85	Standard query 0x6322 A stat.radar.ingmail.ru OPT
498	0.7260450988	172.28.126.227	172.234.216.2	DNS	265	Standard query response 0x6322 A stat.radar.ingmail.ru CNAME common.radar.ingmail.ru A 95.163.58.222 NS bns1.mail.ru NS bns2.mail.ru A...
499	0.7260450988	172.28.126.227	172.234.216.2	DNS	152	Standard query response 0x6322 A stat.radar.ingmail.ru SOA bns1.mail.ru OPT
500	0.7260450988	172.28.126.227	172.234.216.2	DNS	88	Standard query 0x6322 A stat.radar.ingmail.ru OPT
501	0.7260450988	172.28.126.227	172.234.216.2	DNS	87	Standard query 0x6322 A home.ingmail.ru OPT
502	0.7260450988	172.28.126.227	172.234.216.2	DNS	87	Standard query 0x6322 A home.ingmail.ru OPT
503	0.7260450988	172.28.126.227	172.234.216.2	DNS	169	Standard query response 0x6322 A home.ingmail.ru CNAME e-mail.ru.cdn-vk.ru SOA ns1.cdn-vk.ru OPT
504	0.7260450988	172.28.126.227	172.234.216.2	DNS	188	Standard query response 0x6322 A home.ingmail.ru CNAME e-mail.ru.cdn-vk.ru A 91.231.134.1 NS ns2.cdn-vk.ru NS ns1.cdn-vk.ru OPT
505	0.7260450988	172.28.126.227	172.234.216.2	DNS	177	Standard query response 0x6322 A home.ingmail.ru CNAME home.cdn-vk.ru A 91.231.134.1 NS ns2.cdn-vk.ru NS ns1.cdn-vk.ru OPT
506	0.7260450988	172.28.126.227	172.234.216.2	DNS	163	Standard query response 0x6322 A home.ingmail.ru CNAME home.cdn-vk.ru SOA ns1.cdn-vk.ru OPT
507	0.7260450988	172.28.126.227	172.234.216.2	DNS	85	Standard query 0x6322 A home.ingmail.ru OPT
508	0.7260450988	172.28.126.227	172.234.216.2	DNS	140	Standard query response 0x6322 A home.ingmail.ru SOA ns1.cdn-vk.ru OPT
509	0.7260450988	172.28.126.227	172.234.216.2	DNS	135	Standard query response 0x6322 A home.ingmail.ru SOA ns1.cdn-vk.ru OPT
510	0.7260450988	172.28.126.227	172.234.216.2	DNS	83	Standard query 0x6322 A rs.mail.ru OPT
511	0.7260450988	172.28.126.227	172.234.216.2	DNS	81	Standard query 0x6322 A rs.mail.ru OPT
512	0.7260450988	172.28.126.227	172.234.216.2	DNS	81	Standard query 0x6322 A rs.mail.ru OPT
513	0.7260450988	172.28.126.227	172.234.216.2	DNS	149	Standard query response 0x6322 A rs.mail.ru CNAME r.mail.ru SOA bns1.mail.ru OPT
514	0.7260450988	172.28.126.227	172.234.216.2	DNS	80	Standard query 0x6322 A rs.mail.ru OPT
515	0.7260450988	172.28.126.227	172.234.216.2	DNS	149	Standard query response 0x6322 A rs.mail.ru CNAME r.mail.ru SOA bns1.mail.ru OPT
516	0.7260450988	172.28.126.227	172.234.216.2	DNS	132	Standard query response 0x6322 A rs.mail.ru SOA bns1.mail.ru OPT

Рисунок 9 – Список перехваченных пакетов

На рисунке 9 представлен список перехваченных DNS-пакетов. В поле **Info** видны запросы вида:

- *Standard query ... HTTPS mail.ru*
- *Standard query ... A mail.ru*
- *Standard query response ... A 89.221.239.1*
- *... SOA*
- *... CNAME*

Также видно, что браузер посылает запросы **не только** для **mail.ru**, но и для связанных доменов:

- login.vk.ru
- account.mail.ru
- stat.radar.imgsmail.ru
- CDN-домены (cdn-vk.ru, imgsmail.ru и т.д.)

Это подтверждает: для каждой отдельной подгруженной сущности (страницы, картинок, API, статики) браузеру может потребоваться **отдельный DNS-запрос**.

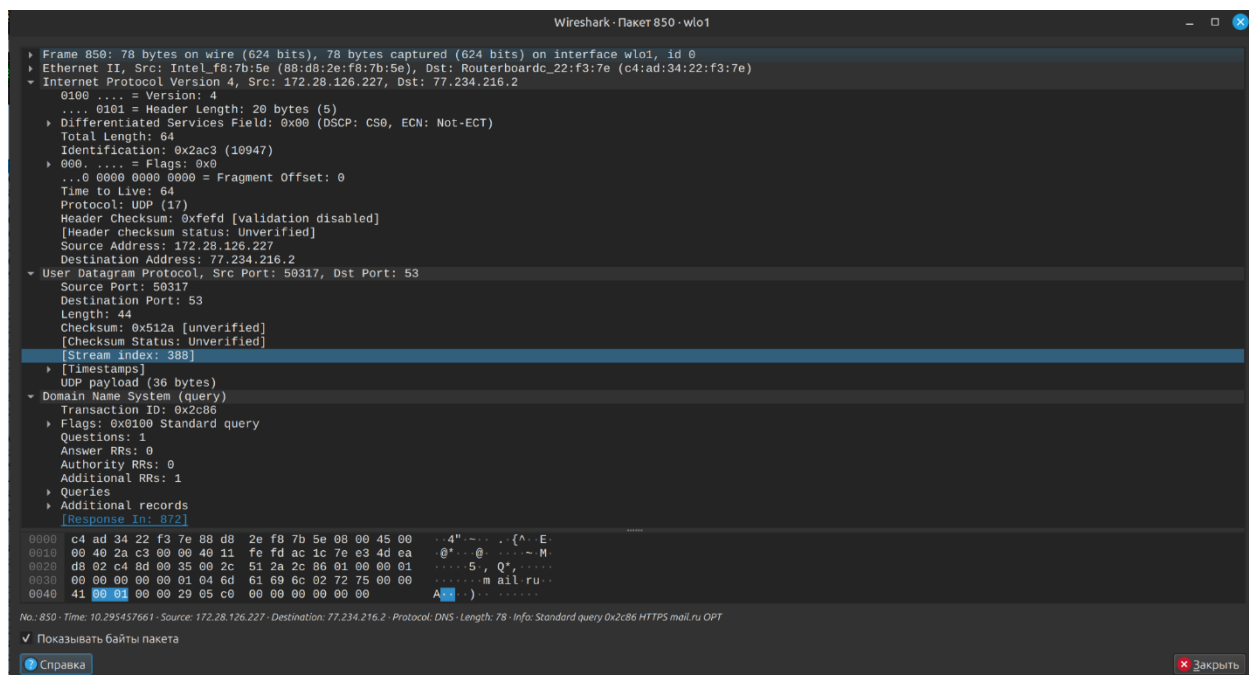


Рисунок 10 – DNS-пакет

На рисунке 10 показан DNS-пакет, отправленный от моего ПК (172.28.126.227) на DNS-сервер провайдера (77.234.216.2).

В раскрытой структуре видно:

- Transaction ID — уникальный идентификатор запроса

- Flags — **Standard Query**
- Questions — **1 вопрос**
- **Name: mail.ru**
- **Type: A (IPv4 address)**

Это означает, что мой компьютер спрашивает у DNS-сервера «Какой IPv4-адрес соответствует доменному имени mail.ru?».

DNS-запрос отправляется **не на mail.ru**, а на DNS-сервер провайдера — поэтому адрес назначения пакета отличается от адреса запрашиваемого сайта.

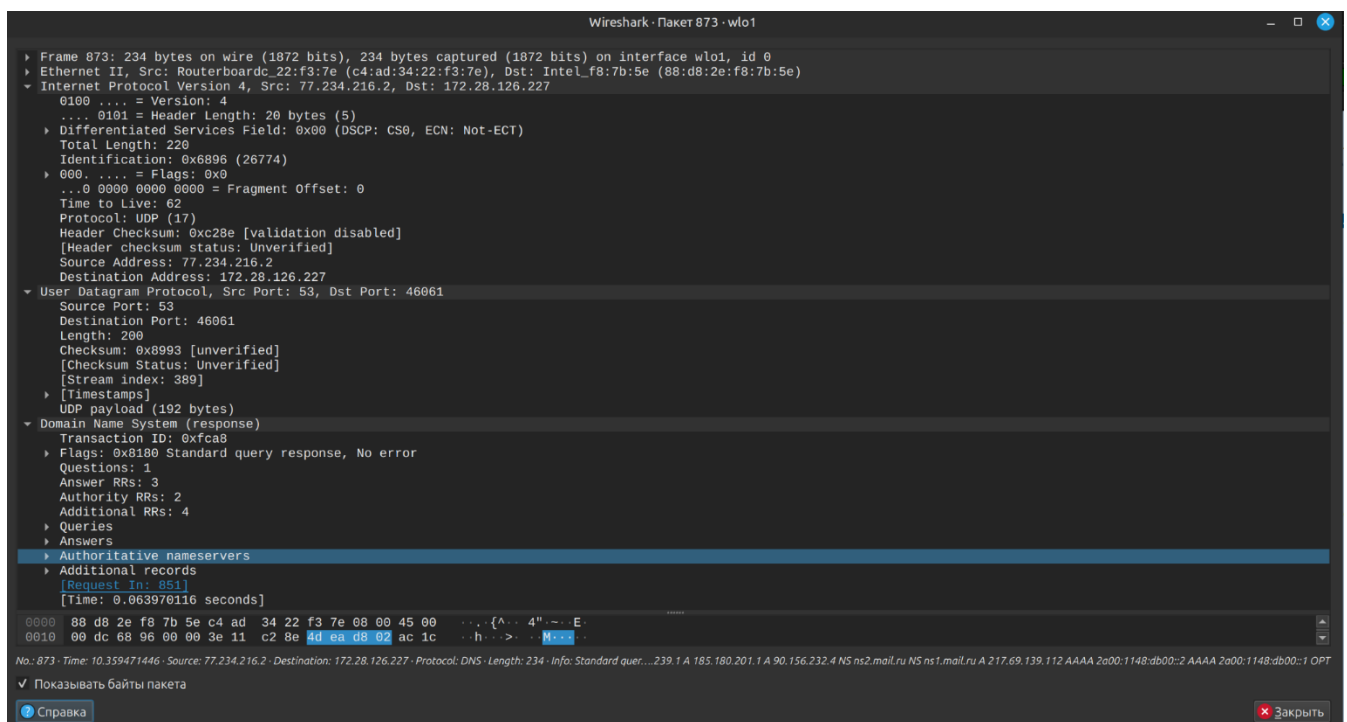


Рисунок 11 – Ответ на запрос

На рисунке 11 показан соответствующий ответ DNS-сервера.

Структура пакета содержит:

- Flags: **0x8180 (Standard query response, No error)**
- Answer RRs: **3 записи**
- Authority RRs: **2 записи**
- Additional RRs: **4 записи**

В списке **Answer Records** видны реальные IP-адреса mail.ru, например:

- 89.221.239.1
- 185.180.201.1
- 90.156.232.4

Также присутствуют:

- **AAAA-записи** — адреса IPv6
- **NS-записи (nameserver)** — ns1.mail.ru, ns2.mail.ru
- информация SOA и OPT

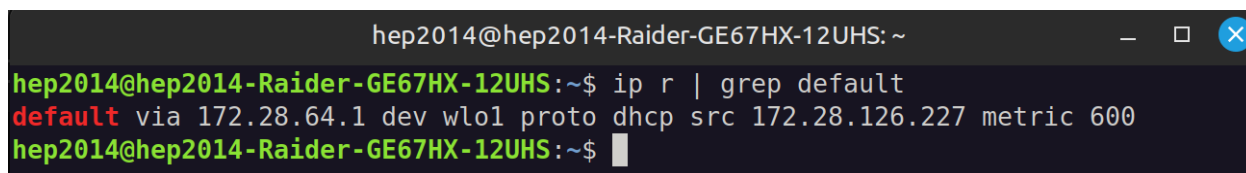
Это демонстрирует, что DNS-ответы содержат не только IP-адреса, но и структурные данные о домене, сведения о серверах, ответственных за зону, дополнительные данные для оптимизации клиентского кэширования.

Выводы по анализу.

1. Адрес DNS-сервера не совпадает с адресом сайта, поскольку браузер не знает IP mail.ru заранее и обращается не к сайту, а к серверу, разрешающему имена;
2. DNS-ответ содержит больше, чем просто IP: A-записи, AAAA-записи, NS-записи, SOA, дополнительные OPT-поля;
3. В трафике видно, что браузер генерирует много отдельных DNS-запросов, потому что страницы mail.ru подгружают контент с разных доменов (vk.ru, cdn-vk.ru, imgsmail.ru и др.). Это подтверждает: если изображения / скрипты находятся на отдельных доменах, для них нужен отдельный DNS-запрос;
4. Механизм работы DNS хорошо виден в Wireshark за счёт структуры пакетов: — Query → Response с Answer / Authority / Additional RRs.

1.5 Анализ протокола ARP

На первом этапе необходимо определить, как устроен выход в сеть на рассматриваемой машине. На рисунке 12 приведён вывод команды `ip r | grep default`. В маршрутизаторской записи видно, что в качестве шлюза по умолчанию используется узел с IP-адресом 172.28.64.1, а исходящие пакеты от имени данной станции отправляются с адреса 172.28.126.227 по интерфейсу `wlo1`. Этот скриншот фиксирует исходные параметры эксперимента: локальный IP, адрес шлюза и используемый интерфейс.



```

hep2014@hep2014-Raider-GE67HX-12UHS: ~
hep2014@hep2014-Raider-GE67HX-12UHS:~$ ip r | grep default
default via 172.28.64.1 dev wlo1 proto dhcp src 172.28.126.227 metric 600
hep2014@hep2014-Raider-GE67HX-12UHS:~$

```

Рисунок 12 – Вывод `ip r`

Далее была очищена таблица ARP/Neighbor, чтобы исключить влияние ранее кэшированных соответствий IP–MAC. На рисунке 13 показан вывод команды `sudo ip -s -s neigh flush all`. В верхней части видно, какие записи присутствовали в таблице до очистки:

для разных IP-адресов подсети 172.28.x.x перечислены MAC-адреса и состояние записей (STALE, REACHABLE). В нижней части отчёта команда сообщает о завершении очистки: «Round 1, deleting 12 entries ... Flush is complete after 1 round». Это означает, что при следующей попытке обращения к шлюзу или к любому другому узлу локального сегмента стек протоколов будет вынужден инициировать новые ARP-запросы.

```

hep2014@hep2014-Raider-GE67HX-12UHS:~$ sudo ip -s -s neigh flush all
[sudo] пароль для hep2014:
172.28.68.9 dev wlo1 lladdr 84:c5:a6:5c:f9:45 used 103/103/76probes 1 STALE
172.28.68.159 dev wlo1 lladdr 58:1c:f8:92:de:9d ref 1 used 11/11/11probes 1 REACHABLE
172.28.123.92 dev wlo1 lladdr 74:4c:a1:69:aa:6b used 162/162/135probes 4 STALE
172.28.68.102 dev wlo1 lladdr e4:c7:67:e8:fe:3b used 192/192/165probes 4 STALE
172.28.68.130 dev wlo1 lladdr b0:be:83:0c:f6:71 used 133/133/107probes 4 STALE
172.28.65.37 dev wlo1 lladdr dc:46:28:24:e2:30 used 109/109/82probes 4 STALE
172.28.64.100 dev wlo1 lladdr f8:9e:94:be:8a:75 used 97/97/71probes 4 STALE
172.28.64.224 dev wlo1 lladdr e8:65:38:ec:22:b3 used 88/88/61probes 1 STALE
172.28.65.198 dev wlo1 lladdr a8:8f:d9:4d:60:d4 used 149/149/122probes 5 STALE
172.28.126.218 dev wlo1 lladdr 14:5a:fc:6b:26:63 used 82/82/55probes 4 STALE
172.28.67.167 dev wlo1 lladdr ec:22:57:64:a9:d7 used 126/126/99probes 4 STALE
172.28.64.1 dev wlo1 lladdr c4:ad:34:22:f3:7e ref 1 used 194/9/194probes 4 REACHABLE

*** Round 1, deleting 12 entries ***
*** Flush is complete after 1 round ***

```

Рисунок 13 – Очистка кэша

После очистки ARP-кэша был запущен обмен ICMP-пакетами с соседними узлами (через ping), параллельно в Wireshark включён захват с фильтром arp. На рисунке 14 представлен общий вид ARP-трафика. Здесь каждая строка в колонке Info содержит человекочитаемое описание типа «Who has 172.28.67.67? Tell 172.28.68.154» или «ARP Announcement for 172.28.108.254». Очень важно, что отправителем запросов обычно является конкретная машина в подсети, а адрес назначения — широковещательный Broadcast. Это хорошо видно: в колонках Source и Destination мелькают различные MAC-адреса рабочих станций и коммутаторов, а протокол везде помечен как ARP. Такой скрин иллюстрирует широковещательный характер ARP-запросов и то, как протокол используется сразу многими устройствами в сегменте.

12990	163.121882689	32:eb:48:ce:64:07	Broadcast	ARP	56 Who has 172.28.67.67? Tell 172.28.68.154
12991	163.121882689	LiteonTechno.69:aa:..	Intel_f8:7b:5e	ARP	56 172.28.123.92 is at 74:4c:a1:69:aa:6b
12993	163.127295234	Intel_6c:b8:17	Intel_f8:7b:5e	ARP	56 172.28.68.37 is at 6c:2f:80:6c:b8:17
12995	163.138779505	96:30:d2:47:e7:7e	Broadcast	ARP	56 Who has 172.28.125.150? Tell 172.28.66.81
12997	163.147127279	96:59:f1:05:31:64	Broadcast	ARP	56 Who has 172.28.127.203? Tell 172.28.123.16
12998	163.149655403	d2:fd:f6:da:c7:00	Broadcast	ARP	56 Who has 172.28.124.159? Tell 172.28.126.217
13001	163.164890195	Intel_45:32:e7	Broadcast	ARP	56 Who has 172.28.64.1? Tell 172.28.121.240
13004	163.176357135	ea:94:1a:84:ff:de	Broadcast	ARP	56 Who has 172.28.68.22? Tell 172.28.125.41
13009	163.188610649	96:59:f1:05:31:64	Broadcast	ARP	56 Who has 172.28.123.40? Tell 172.28.123.16
13012	163.195882126	e2:09:10:1d:95:33	Broadcast	ARP	56 Who has 172.28.65.211? Tell 172.28.127.155
13024	163.242955990	2e:c0:99:51:f8:25	Broadcast	ARP	56 Who has 172.28.65.211? Tell 172.28.124.187
13034	163.551530342	C2120F60B0D2070	Broadcast	ARP	56 Who has 172.28.65.211? Tell 172.28.123.129
13035	163.551530356	b2:df:30:f9:6d:2b	Broadcast	ARP	56 ARP Announcement for 172.28.108.254
13050	163.601497786	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.123.214? Tell 172.28.108.254
13051	163.601497782	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.67.223? Tell 172.28.108.254
13052	163.601497810	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.65.147? Tell 172.28.108.254
13053	163.601497835	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.126.107? Tell 172.28.108.254
13054	163.601497863	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.65.190? Tell 172.28.108.254
13055	163.601497891	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.68.130? Tell 172.28.108.254
13056	163.601497918	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.125.67? Tell 172.28.108.254
13057	163.601497947	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.68.180? Tell 172.28.108.254
13058	163.601520653	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.119.180? Tell 172.28.108.254
13059	163.601520692	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.127.79? Tell 172.28.108.254
13060	163.601520732	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.124.37? Tell 172.28.108.254
13061	163.606525646	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.123.16? Tell 172.28.108.254
13062	163.611768498	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.123.200? Tell 172.28.108.254
13063	163.611768726	b2:df:30:f9:6d:2b	Broadcast	ARP	56 Who has 172.28.127.88? Tell 172.28.108.254

Рисунок 14 – Представлен общий вид ARP

Более детально структура ARP-запроса показана на рисунке 15. Здесь выбран один из кадров типа request; в верхней части окна Wireshark отображается заголовок канального уровня Ethernet II: источником является узел с MAC-адресом 32:e8:48:ce:64:07, а адрес назначения — ff:ff:ff:ff:ff:ff, то есть широковещательный. Ниже, в блоке Address Resolution Protocol, видно, что протокол работает поверх IPv4 (значение поля Protocol type — 0x0800), аппаратный тип — Ethernet, длина аппаратного и сетевого адреса составляет 6 и 4 байта соответственно. Поле Opcode имеет значение 1, что однозначно указывает на запрос (request). В полях Sender MAC address и Sender IP address находятся реальные реквизиты инициатора, тогда как Target MAC address заполнен нулями, а в Target IP address записан интересующий адрес 172.28.67.67. Такое сочетание значений означает, что отправитель знает IP-адрес целевого узла, но не знает его аппаратный адрес и потому вынужден обратиться ко всем участникам локального сегмента с вопросом «кто владеет этим IP?».

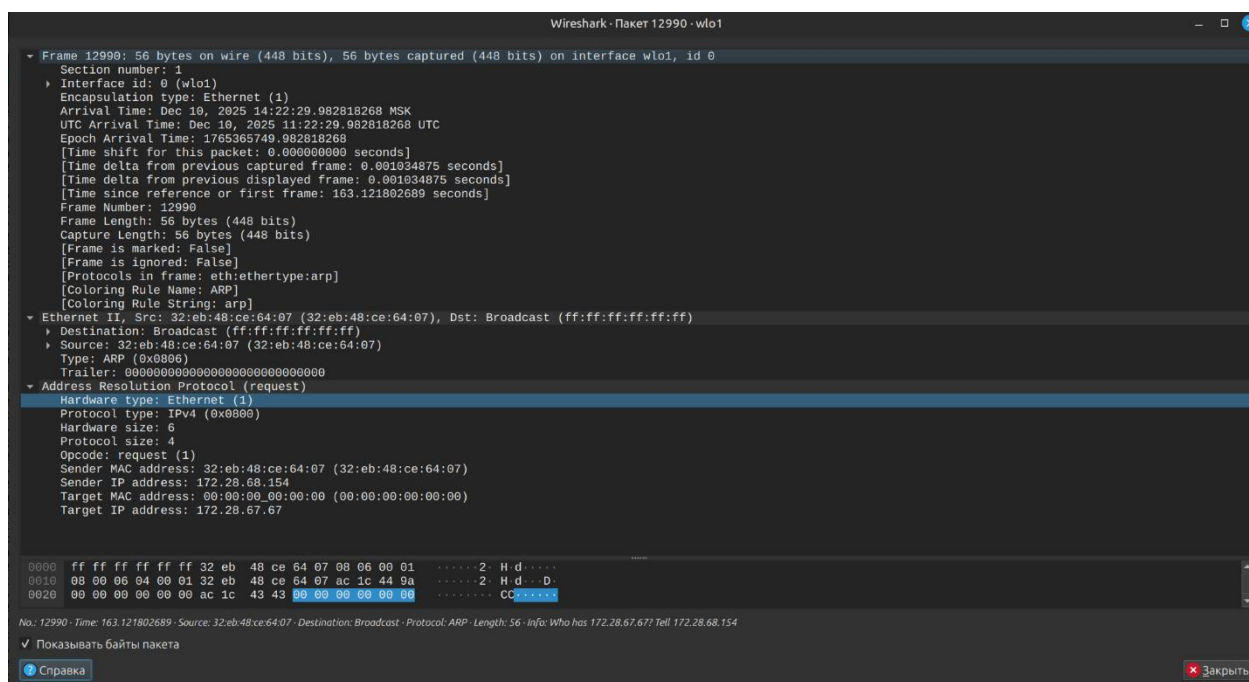


Рисунок 15 – Структура ARP-запроса

На рисунке 16 показан ответный кадр протокола ARP. Здесь уже нет широковещания: в заголовке Ethernet II источником является устройство с MAC-адресом 74:4c:a1:69:aa:6b (подпись Wireshark — LiteonTechno_69:aa:6b), назначением — конкретный MAC-адрес моей рабочей станции 88:d8:2e:f8:7b:5e (Intel_f8:7b:5e). В блоке Address Resolution Protocol поле Opcode имеет значение 2, то есть это reply. В Sender IP address записан IP-адрес 172.28.123.92, а в Sender MAC address — тот же 74:4c:a1:69:aa:6b. Соответственно, эта станция заявляет: «адрес 172.28.123.92 принадлежит мне, мой MAC — 74:4c:a1:69:aa:6b». Поля Target IP address и Target MAC address содержат IP и MAC

инициатора запроса: 172.28.126.227 и 88:d8:2e:f8:7b:5e. Такой кадр уже не рассылается всем, он направлен строго тому хосту, который задавал вопрос, и даёт ему готовое соответствие IP–MAC для последующей записи в локальный ARP-кэш.

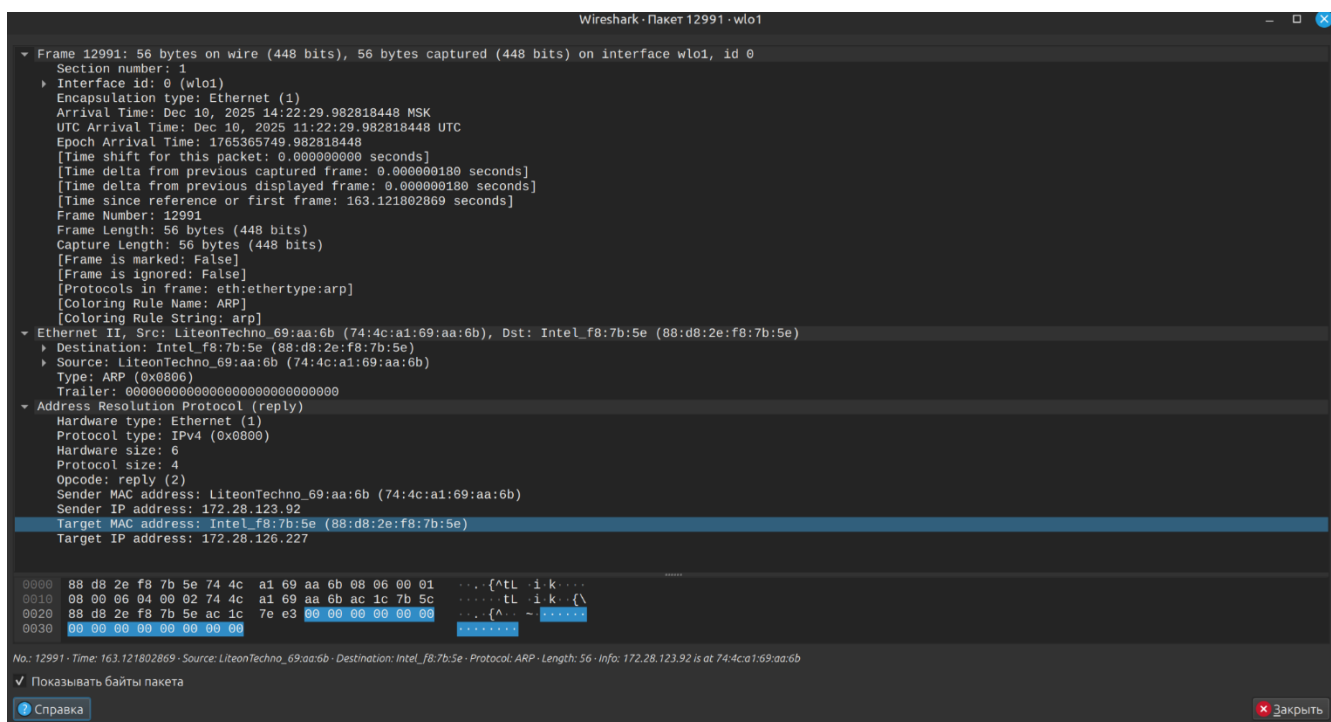


Рисунок 16 – Ответ на запрос

Выводы по анализу.

В совокупности пять скриншотов демонстрируют полный цикл работы ARP. Сначала по выводу маршрутов и содержимому таблицы соседей устанавливаются реальные параметры сети и очищается кэш. Затем Wireshark фиксирует широковещательный запрос на уровне канального протокола Ethernet с нулевым целевым MAC-адресом и заполненным целевым IP. После этого приходящий ответ несёт в себе окончательное сопоставление IP-адреса и аппаратного адреса устройства. Получив такой reply, операционная система добавляет запись в ARP-таблицу и может в дальнейшем передавать IP-пакеты этому узлу, формируя кадры Ethernet сразу с правильным MAC-адресом. Таким образом на практике подтверждается, что ARP является критически важным служебным протоколом, обеспечивающим связку сетевого уровня (IP) и канального уровня (Ethernet) внутри локального сегмента.

1.6 Анализ dns-пакетов с помощью утилиты nslookup

На первом этапе был выполнен базовый DNS-запрос для определения IPv4-адресов доменного имени mail.ru. На рисунке 17 приведён вывод команды:

```
nslookup mail.ru.
```

В верхней части вывода указано, что в качестве DNS-сервера используется локальный stub-резолвер systemd-resolved с адресом 127.0.0.53, принимающий запросы приложений и пересылающий их далее вышестоящему DNS-серверу.

В блоке Non-authoritative answer представлены несколько IPv4-адресов (89.221.239.1, 185.180.201.1, 90.156.232.4), соответствующих доменному имени mail.ru. Наличие нескольких адресов свидетельствует о применении механизмов балансировки нагрузки и распределения запросов между серверами инфраструктуры домена.

```
hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup mail.ru
Server:           127.0.0.53
Address:          127.0.0.53#53

Non-authoritative answer:
Name:   mail.ru
Address: 89.221.239.1
Name:   mail.ru
Address: 185.180.201.1
Name:   mail.ru
Address: 90.156.232.4
```

Рисунок 17 – Вывод команды

Далее был выполнен анализ DNS-трафика в среде Wireshark. Захват пакетов осуществлялся на активном сетевом интерфейсе с установленным фильтром dns. На рисунке 18 показан общий вид DNS-обмена, связанного с выполнением команды nslookup mail.ru. В списке пакетов отчётливо видны стандартный DNS-запрос типа A и ответ DNS-сервера, содержащий несколько ресурсных записей с IPv4-адресами.

No.	Time	Source	Destination	Protocol	Length	Info
26	4.849368036	192.168.0.89	192.168.0.1	DNS	67	Standard query 0x2859 A mail.ru
27	4.853821538	192.168.0.1	192.168.0.89	DNS	115	Standard query response 0x2859 A mail.ru A 90.156.232.4 A 185.180.201.1 A 89.221.239.1
28	4.854649613	192.168.0.89	192.168.0.1	DNS	67	Standard query 0x9d28 AAAA mail.ru
29	4.857489924	192.168.0.1	192.168.0.89	DNS	118	Standard query response 0x9d28 AAAA mail.ru SOA ns1.mail.ru

Рисунок 18 – Общий вид

Для проверки поддержки IPv6 был выполнен запрос ресурсных записей типа AAAA. На рисунке 19 приведён вывод команды:

```
nslookup -type=AAAA mail.ru
```

DNS-сервер не вернул IPv6-адресов, о чём свидетельствует сообщение No answer.

Анализ соответствующих пакетов в Wireshark показывает, что в ответе присутствует запись SOA (Start of Authority), указывающая на авторитетный DNS-сервер зоны mail.ru. Такой ответ является корректным и означает отсутствие AAAA-записей, то есть домен mail.ru в данной конфигурации не использует IPv6-адресацию.

```

hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup -type=AAAA mail.ru
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
*** Can't find mail.ru: No answer

```

185	42.435453279	192.168.0.89	192.168.0.1	DNS	76 Standard query 0xc633 A mail.ru OPT
186	42.439892561	192.168.0.1	192.168.0.89	DNS	126 Standard query response 0xc633 A mail.ru A 89.221.239.1 A 185.188.281.1 A 98.156.232.4 OPT
187	42.441644099	192.168.0.89	192.168.0.1	DNS	78 Standard query 0x9aaf AAAA mail.ru OPT
188	42.467226181	192.168.0.1	192.168.0.89	DNS	129 Standard query response 0x9aaf AAAA mail.ru SOA ns1.mail.ru OPT
234	52.797973528	192.168.0.89	192.168.0.1	DNS	91 Standard query 0xb158 HTTPS a.nel.cloudflare.com OPT
235	52.797275390	192.168.0.1	192.168.0.89	DNS	91 Standard query 0x750c A a.nel.cloudflare.com OPT
236	52.880813798	192.168.0.89	192.168.0.1	DNS	142 Standard query response 0x0158 HTTPS a.nel.cloudflare.com SOA coleman.ns.cloudflare.com OPT
237	52.880893481	192.168.0.1	192.168.0.89	DNS	107 Standard query response 0x750c A a.nel.cloudflare.com A 35.190.80.1 OPT
272	57.201625784	192.168.0.89	192.168.0.1	DNS	78 Standard query 0xfd8e AAAA mail.ru OPT
273	57.204197663	192.168.0.1	192.168.0.89	DNS	129 Standard query response 0xfd8e AAAA mail.ru SOA ns1.mail.ru OPT
284	72.404635059	192.168.0.89	192.168.0.1	DNS	108 Standard query 0x183c A connectivity-check.ubuntu.com OPT
285	72.409459968	192.168.0.1	192.168.0.89	DNS	292 Standard query response 0x183c A connectivity-check.ubuntu.com A 185.125.190.49 A 91.189.91.98 A 185.125.190.97 A 91.189.91.48 A 185.125.190.97
327	80.217585159	192.168.0.89	192.168.0.1	DNS	78 Standard query 0xb248 MX mail.ru OPT
328	80.237680739	192.168.0.1	192.168.0.89	DNS	98 Standard query response 0xb248 MX mail.ru MX 10 mxs.mail.ru OPT
533	126.368711225	192.168.0.89	192.168.0.1	DNS	78 Standard query 0xf9fc NS mail.ru OPT
534	126.373452443	192.168.0.1	192.168.0.89	DNS	114 Standard query response 0xf9fc NS mail.ru NS ns1.mail.ru NS ns2.mail.ru OPT
576	136.141304473	192.168.0.89	192.168.0.1	DNS	78 Standard query 0x13d6 TXT mail.ru OPT
577	136.150181863	192.168.0.1	192.168.0.89	DNS	206 Standard query response 0x13d6 TXT mail.ru TXT TXT OPT
686	178.074458374	192.168.0.89	192.168.0.1	DNS	86 Standard query 0x079b A ssl.gstatic.com OPT
687	178.079313233	192.168.0.1	192.168.0.89	DNS	143 Standard query response 0xf9b1 HTTPS ssl.gstatic.com SOA ns1.google.com OPT
688	178.079313297	192.168.0.89	192.168.0.1	DNS	102 Standard query response 0x079b A ssl.gstatic.com A 74.125.131.94 OPT

Рисунок 19 – Вывод команды

Следующим этапом был выполнен запрос MX-записей, определяющих почтовую маршрутизацию домена. На рисунке 20 представлен вывод команды:

```
nslookup -type=MX mail.ru
```

Из результата следует, что почтовые сообщения для домена mail.ru направляются на сервер mxs.mail.ru с приоритетом 10. Это означает, что указанный сервер является основным почтовым шлюзом домена.

```

hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup -type=MX mail.ru
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
mail.ru mail exchanger = 10 mxs.mail.ru.

Authoritative answers can be found from:

```

Рисунок 20 – Вывод команды

Для определения авторитетных DNS-серверов домена был выполнен запрос NS-записей. На рисунке 21 показан вывод команды:

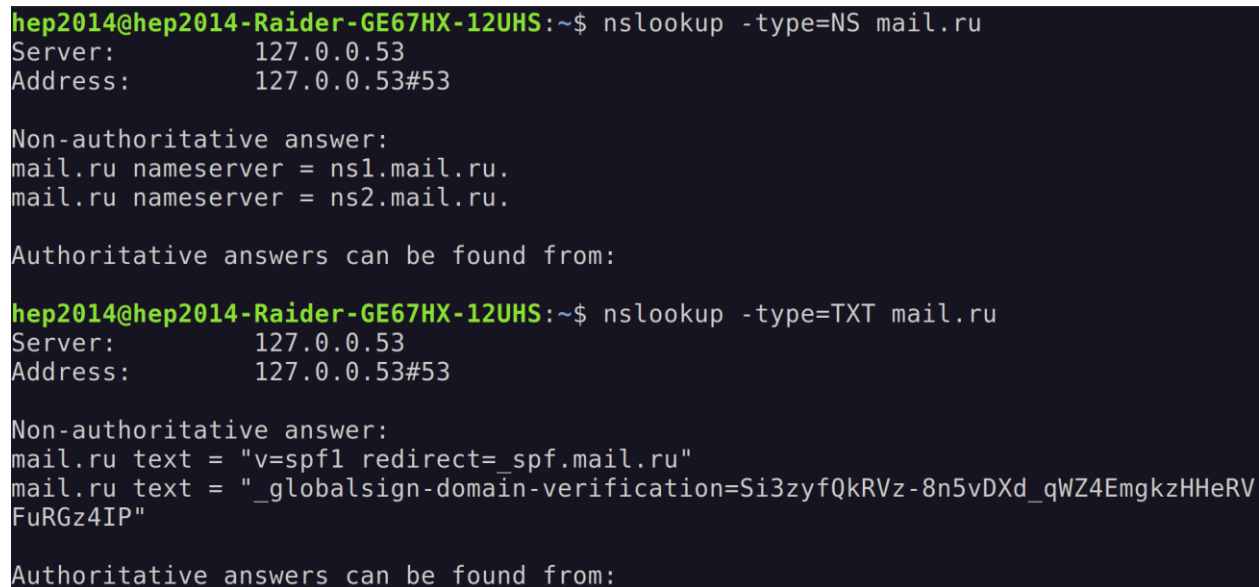
```
nslookup -type=NS mail.ru
```

В результате были получены имена серверов ns1.mail.ru и ns2.mail.ru, которые отвечают за хранение и обслуживание DNS-зоны данного домена.

Затем был выполнен анализ TXT-записей домена. На рисунке 21 приведён результат выполнения команды:

```
nslookup -type=TXT mail.ru
```

В ответе содержатся служебные текстовые записи, включая SPF-политику (v=spf1 redirect=_spf.mail.ru), используемую для защиты почтового домена от подмены отправителя, а также запись проверки владения доменом, применяемую при выпуске цифровых сертификатов.



```
hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup -type=NS mail.ru
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
mail.ru nameserver = ns1.mail.ru.
mail.ru nameserver = ns2.mail.ru.

Authoritative answers can be found from:

hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup -type=TXT mail.ru
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
mail.ru text = "v=spf1 redirect=_spf.mail.ru"
mail.ru text = "_globalsign-domain-verification=Si3zyfQkRVz-8n5vDXd_qWZ4EmgkzHHeRVFuRGz4IP"

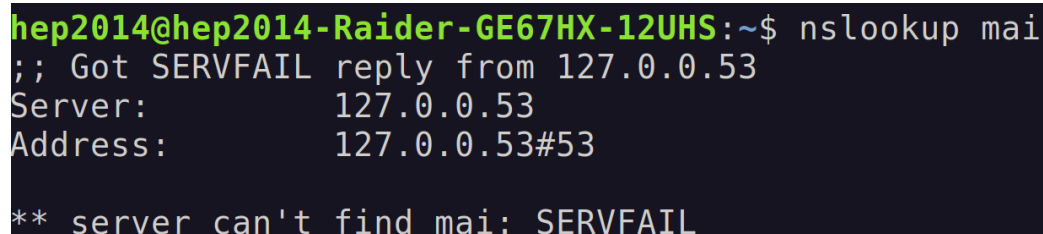
Authoritative answers can be found from:
```

Рисунок 21 – Вывод команд

Отдельно была выполнена попытка DNS-разрешения имени mai, не являющегося полным доменным именем. На рисунке 23 показан результат выполнения команды:

```
nslookup mai
```

завершившийся ошибкой SERVFAIL. Это демонстрирует, что система доменных имён не поддерживает поиск по подстроке и обрабатывает исключительно полностью квалифицированные доменные имена.



```
hep2014@hep2014-Raider-GE67HX-12UHS:~$ nslookup mai
;; Got SERVFAIL reply from 127.0.0.53
Server:          127.0.0.53
Address:         127.0.0.53#53

** server can't find mai: SERVFAIL
```

Рисунок 22 – Вывод команды

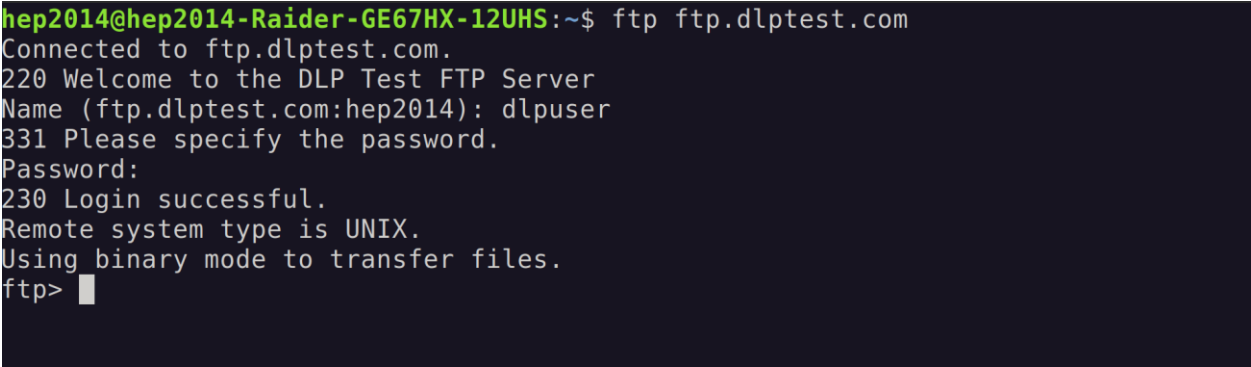
Выводы по анализу.

В ходе выполнения анализа трафика утилиты nslookup был исследован процесс DNS-разрешения доменных имён на практике. Экспериментально подтверждено, что при обращении к домену формируются стандартные DNS-запросы и ответы, содержащие ресурсные записи различных типов (A, MX, NS, TXT). Также показано, что отсутствие AAAA-записей корректно отражается в виде SOA-ответа, а поиск по неполному имени невозможен в рамках DNS. Захват пакетов в Wireshark наглядно демонстрирует клиент-серверную природу DNS и роль локального резолвера systemd-resolved в процессе разрешения имён.

1.7 Анализ FTP-трафика

На первом этапе было выполнено установление FTP-соединения с тестовым сервером ftp.dlptest.com. Для этого в терминале была использована команда ftp ftp.dlptest.com. На рисунке 23 показан процесс подключения к серверу и начало диалога по управляющему каналу FTP. Сервер отвечает кодом 220 Welcome, что свидетельствует о готовности принять соединение. Далее клиенту предлагается ввести имя пользователя.

После установления соединения была выполнена аутентификация пользователя. В качестве имени пользователя был указан dlpuser, после чего сервер запросил пароль (331 Please specify the password). После ввода пароля сервер вернул ответ 230 Login successful, подтверждающий успешную авторизацию. Важно отметить, что команды USER и PASS передаются в незашифрованном виде, что позволяет наблюдать их содержимое при анализе трафика.



```
hep2014@hep2014-Raider-GE67HX-12UHS:~$ ftp ftp.dlptest.com
Connected to ftp.dlptest.com.
220 Welcome to the DLP Test FTP Server
Name (ftp.dlptest.com:hep2014): dlpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> █
```

Рисунок 23 – Успешное подключение

После успешной аутентификации был выполнен просмотр содержимого текущего каталога на FTP-сервере с помощью команды ls. На рисунке 25 приведён вывод данной команды в терминале, содержащий список файлов, расположенных в удалённом каталоге.

При выполнении команды ls по управляющему каналу отправляется команда LIST, после чего сервер инициирует передачу данных по отдельному каналу данных в пассивном режиме.

```
ftp> ls
229 Entering Extended Passive Mode (|||1117|).
150 Here comes the directory listing.
-rw-r--r-- 1 1001 1001 140563 Dec 17 18:20 10.101.1.2 20251217-13205720_IVA.jpg
-rw-r--r-- 1 1001 1001 142238 Dec 17 18:20 10.101.1.2 20251217-13210402_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:20 10.101.1.2 20251217-13210659_IVA.jpg
-rw-r--r-- 1 1001 1001 142194 Dec 17 18:20 10.101.1.2 20251217-13211260_IVA.jpg
-rw-r--r-- 1 1001 1001 115099 Dec 17 18:20 10.101.1.2 20251217-13211361_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:20 10.101.1.2 20251217-13211521_IVA.jpg
-rw-r--r-- 1 1001 1001 125609 Dec 17 18:20 10.101.1.2 20251217-13212202_IVA.jpg
-rw-r--r-- 1 1001 1001 129076 Dec 17 18:20 10.101.1.2 20251217-13212982_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:20 10.101.1.2 20251217-13214203_IVA.jpg
-rw-r--r-- 1 1001 1001 128555 Dec 17 18:21 10.101.1.2 20251217-13215341_IVA.jpg
-rw-r--r-- 1 1001 1001 127700 Dec 17 18:21 10.101.1.2 20251217-13215562_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13215943_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13220682_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13221342_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13221882_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13222483_IVA.jpg
-rw-r--r-- 1 1001 1001 130663 Dec 17 18:21 10.101.1.2 20251217-13223261_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:21 10.101.1.2 20251217-13224063_IVA.jpg
-rw-r--r-- 1 1001 1001 134436 Dec 17 18:22 10.101.1.2 20251217-13224623_IVA.jpg
-rw-r--r-- 1 1001 1001 132329 Dec 17 18:22 10.101.1.2 20251217-13225583_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:22 10.101.1.2 20251217-13225903_IVA.jpg
-rw-r--r-- 1 1001 1001 131685 Dec 17 18:22 10.101.1.2 20251217-13230362_IVA.jpg
-rw-r--r-- 1 1001 1001 133934 Dec 17 18:22 10.101.1.2 20251217-13231242_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:22 10.101.1.2 20251217-13231684_IVA.jpg
-rw-r--r-- 1 1001 1001 132764 Dec 17 18:22 10.101.1.2 20251217-13232003_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:22 10.101.1.2 20251217-13232124_IVA.jpg
-rw-r--r-- 1 1001 1001 132230 Dec 17 18:22 10.101.1.2 20251217-13232703_IVA.jpg
-rw-r--r-- 1 1001 1001 133934 Dec 17 18:22 10.101.1.2 20251217-13233922_IVA.jpg
-rw-r--r-- 1 1001 1001 131866 Dec 17 18:22 10.101.1.2 20251217-13234184_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:22 10.101.1.2 20251217-13234463_IVA.jpg
-rw-r--r-- 1 1001 1001 133944 Dec 17 18:23 10.101.1.2 20251217-13235225_IVA.jpg
-rw-r--r-- 1 1001 1001 132325 Dec 17 18:23 10.101.1.2 20251217-13235384_IVA.jpg
-rw-r--r-- 1 1001 1001 131322 Dec 17 18:23 10.101.1.2 20251217-13235504_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:23 10.101.1.2 20251217-13240185_IVA.jpg
-rw-r--r-- 1 1001 1001 0 Dec 17 18:23 10.101.1.2 20251217-13241285_IVA.jpg
-rw-r--r-- 1 1001 1001 125105 Dec 17 18:23 10.101.1.2 20251217-13241907_IVA.jpg
```

Рисунок 24 – Вывод команды ls

На рисунке 25 показан соответствующий фрагмент FTP-трафика в Wireshark. В нём отчётливо видны команды EPSV и LIST, а также ответ сервера 150 Here comes the directory listing и 226 Directory send OK. Это подтверждает использование отдельных каналов управления и передачи данных, характерных для протокола FTP.

No.	Time	Source	Destination	Protocol	Length	Info
8124	307.962576665	44.241.66.173	192.168.0.89	FTP	106	Response: 220 Welcome to the DLP Test FTP Server
8145	312.797819837	192.168.0.89	44.241.66.173	FTP	80	Request: USER dlpuser
8147	312.980086293	44.241.66.173	192.168.0.89	FTP	100	Response: 331 Please specify the password.
8151	318.165146927	192.168.0.89	44.241.66.173	FTP	98	Request: PASS rNkYTX0g7z3RqJ8mXvGhBeu
8157	319.022868799	44.241.66.173	192.168.0.89	FTP	89	Response: 230 Login successful.
8159	319.023131350	192.168.0.89	44.241.66.173	FTP	72	Request: SYST
8162	319.190057010	44.241.66.173	192.168.0.89	FTP	85	Response: 215 UNIX Type: L8
8163	319.190379348	192.168.0.89	44.241.66.173	FTP	72	Request: FEAT
8164	319.433537398	44.241.66.173	192.168.0.89	FTP	81	Response: 211-Features:
8165	319.433537987	44.241.66.173	192.168.0.89	FTP	154	Response: AUTH TLS
8166	319.433538891	44.241.66.173	192.168.0.89	FTP	75	Response: 211 End
8200	321.222165660	192.168.0.89	44.241.66.173	FTP	72	Request: EPSV
8201	321.480032688	44.241.66.173	192.168.0.89	FTP	114	Response: 229 Entering Extended Passive Mode (1084).
8206	321.684382276	192.168.0.89	44.241.66.173	FTP	72	Request: LIST
8207	321.888963473	44.241.66.173	192.168.0.89	FTP	105	Response: 150 Here comes the directory listing.
8214	322.093899350	44.241.66.173	192.168.0.89	FTP	90	Response: 226 Directory send OK.

Рисунок 25 – Фрагмент трафика

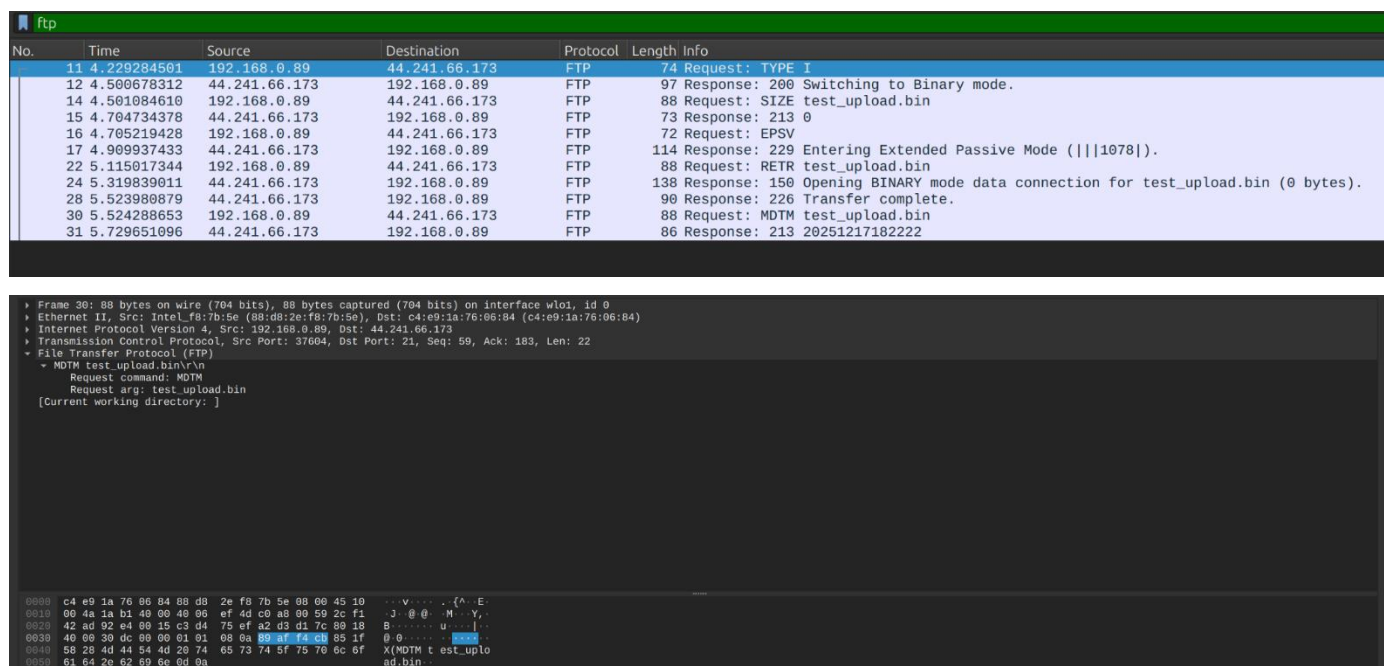
Далее была выполнена загрузка файла с сервера на локальную машину с помощью команды get test_upload.bin. На рисунке 26 представлен вывод данной команды в терминале.

Сервер сообщает о переходе в расширенный пассивный режим (229 Entering Extended Passive Mode), открытии бинарного канала передачи данных (150 Opening BINARY mode data connection) и успешном завершении передачи файла (226 Transfer complete).

```
ftp> get test_upload.bin
local: test_upload.bin remote: test_upload.bin
229 Entering Extended Passive Mode (|||1078|).
150 Opening BINARY mode data connection for test_upload.bin (0 bytes).
0 0.00 KiB/s
226 Transfer complete.
```

Рисунок 26 – Выполнение команды get

Анализ соответствующего трафика в Wireshark показан на рисунке 27. Здесь видны команды RETR test_upload.bin, ответы сервера и последующая передача данных по каналу FTP-DATA. Это демонстрирует, что передача файлов в FTP осуществляется по отдельному TCP-соединению, отличному от управляющего канала.



No.	Time	Source	Destination	Protocol	Length	Info
11	4.220284501	192.168.0.89	44.241.66.173	FTP	74	Request: TYPE I
12	4.500678312	44.241.66.173	192.168.0.89	FTP	97	Response: 200 Switching to Binary mode.
14	4.501084610	192.168.0.89	44.241.66.173	FTP	88	Request: SIZE test_upload.bin
15	4.704734378	44.241.66.173	192.168.0.89	FTP	73	Response: 213 0
16	4.705219428	192.168.0.89	44.241.66.173	FTP	72	Request: EPSV
17	4.909937433	44.241.66.173	192.168.0.89	FTP	114	Response: 229 Entering Extended Passive Mode (1078).
22	5.115017344	192.168.0.89	44.241.66.173	FTP	88	Request: RETR test_upload.bin
24	5.319839011	44.241.66.173	192.168.0.89	FTP	138	Response: 150 Opening BINARY mode data connection for test_upload.bin (0 bytes).
28	5.523980879	44.241.66.173	192.168.0.89	FTP	90	Response: 226 Transfer complete.
30	5.524288653	192.168.0.89	44.241.66.173	FTP	88	Request: MDTM test_upload.bin
31	5.729651096	44.241.66.173	192.168.0.89	FTP	86	Response: 213 20251217182222

Frame 30: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface wlo1, id 0
 Ethernet II, Src: Intel_f8:7b:5e (08:d0:2e:f8:7b:5e), Dst: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84)
 Internet Protocol Version 4, Src: 192.168.0.89, Dst: 44.241.66.173
 Transmission Control Protocol, Src Port: 37604, Dst Port: 21, Seq: 59, Ack: 183, Len: 22
 File Transfer Protocol (FTP)
 MDTM test_upload.bin\r\n
 Request command: MDTM
 Request arg: test_upload.bin
 [Current working directory:]

Рисунок 27 – FTP-трафик при передаче файла

После завершения работы с сервером FTP-сеанс был корректно завершён командой bye. На рисунке 28 показан вывод данной команды в терминале, а сервер отвечает кодом 221 Goodbye, подтверждая закрытие сессии.

```
ftp> bye
221 Goodbye.
```

Рисунок 28 – Завершение сессии

На рисунке 29 приведён соответствующий фрагмент трафика в Wireshark, где отображается команда QUIT и ответ сервера 221 Goodbye. Это демонстрирует корректное завершение соединения по управляющему каналу FTP.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000000000	192.168.0.89	44.241.66.173	FTP	72	Request: QUIT
2	0.254061739	44.241.66.173	192.168.0.89	FTP	80	Response: 221 Goodbye.


```

▶ Frame 1: 72 bytes on wire (576 bits), 72 bytes captured (576 bits) on interface wlo1, id 0
▶ Ethernet II, Src: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e), Dst: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84)
▶ Internet Protocol Version 4, Src: 192.168.0.89, Dst: 44.241.66.173
▼ Transmission Control Protocol, Src Port: 37604, Dst Port: 21, Seq: 1, Ack: 1, Len: 6
  Source Port: 37604
  Destination Port: 21
  [Stream index: 0]
  ▶ [Conversation completeness: Incomplete (28)]
  [TCP Segment Len: 6]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3285480965
  [Next Sequence Number: 7 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 2731790736
  1000 .... = Header Length: 32 bytes (8)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 16384
  [Calculated window size: 16384]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x30cc [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
  TCP payload (6 bytes)
▼ File Transfer Protocol (FTP)
  QUIT\r\n
  Request command: QUIT
  [Current working directory: ]

```

Рисунок 29 – FTP-трафик при завершении соединения

Выводы по анализу.

В ходе анализа FTP-трафика был исследован полный цикл работы протокола FTP: установление соединения, аутентификация пользователя, получение списка файлов, передача файла и корректное завершение сеанса. Экспериментально подтверждено, что FTP использует отдельный управляющий канал и канал передачи данных, а также передаёт команды и учётные данные в открытом виде. Захват трафика в Wireshark наглядно демонстрирует уязвимость классического FTP-протокола и обосновывает необходимость применения защищённых альтернатив, таких как FTPS или SFTP, в реальных сетях.

1.8 Анализ DHCP-трафика

На первом этапе были зафиксированы текущие параметры сетевых интерфейсов исследуемой системы. На рисунке 30 приведён вывод команды:

```
ip addr show
```

Из него видно, что беспроводной интерфейс wlo1 находится в состоянии UP и уже имеет IPv4-адрес 192.168.0.89/24, полученный динамически, а также назначенные IPv6-адреса. Данный скриншот фиксирует исходное состояние сети перед проведением эксперимента по анализу DHCP.

```

hep2014@hep2014-Raider-GE67HX-12UHS:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid lft forever preferred lft forever
    inet6 ::1/128 scope host noprefixroute
        valid lft forever preferred lft forever
2: enp4s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 04:7c:16:32:e3:e9 brd ff:ff:ff:ff:ff:ff
3: wlo1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 88:d8:2e:f8:7b:5e brd ff:ff:ff:ff:ff:ff
    altname wlp0s20f3
    inet 192.168.0.89/24 brd 192.168.0.255 scope global dynamic noprefixroute wlo1
        valid lft 85573sec preferred lft 85573sec
    inet6 fd01::a415:558b:c7be:1262/64 scope global temporary dynamic
        valid lft 281sec preferred_lft 281sec
    inet6 fd01::7372:b991:20fc:7791/64 scope global dynamic mngtmpaddr noprefixroute
        valid lft 281sec preferred_lft 281sec
    inet6 fe80::3778:a299:9d8a:ff68/64 scope link noprefixroute
        valid lft forever preferred_lft forever
4: br-1b7b8d8b881e: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 8e:05:31:05:d8:64 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-1b7b8d8b881e
        valid lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 8e:03:c2:70:3b:8a brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid lft forever preferred_lft forever
6: br-9578aalf52de: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether b6:8f:f0:77:31:8a brd ff:ff:ff:ff:ff:ff
    inet 172.20.0.1/16 brd 172.20.255.255 scope global br-9578aalf52de
        valid lft forever preferred_lft forever

```

Рисунок 30 – Текущие параметры сетевых интерфейсов

Для исключения влияния ранее полученной DHCP-аренды и принудительного запуска полного цикла DHCP-обмена был выполнен отказ от текущего IP-адреса. С этой целью была использована команда:

```
sudo dhclient -r wlo1
```

В результате выполнения команды клиент уведомляет DHCP-сервер об освобождении ранее выданного IP-адреса 192.168.0.89, после чего данный адрес считается свободным и может быть повторно распределён сервером.

На рисунке 31 приведён вывод команды отказа от аренды, в котором отображается завершение работы DHCP-клиента для интерфейса wlo1. После выполнения данной операции сетевой интерфейс временно остаётся без назначенного IPv4-адреса, что создаёт условия для повторного запуска процедуры динамической конфигурации сети.

```

hep2014@hep2014-Raider-GE67HX-12UHS:~$ sudo nmcli device set wlo1 managed no
hep2014@hep2014-Raider-GE67HX-12UHS:~$ nmcli device status

```

DEVICE	TYPE	STATE	CONNECTION
lo	loopback	подключено (внешнее)	lo
br-1b7b8d8b881e	bridge	подключено (внешнее)	br-1b7b8d8b881e
br-9578aalf52de	bridge	подключено (внешнее)	br-9578aalf52de
docker0	bridge	подключено (внешнее)	docker0
enp4s0	ethernet	недоступно	--
p2p-dev-wlo1	wifi-p2p	недоступно	--
wlo1	wifi	не настроенно	--

Рисунок 31 – Отказ от полученного адреса

Для инициирования нового DHCP-обмена был принудительно запущен DHCP-клиент с подробным выводом. На рисунке 32 представлен вывод команды:

```
sudo dhclient -v wlo1
```

В терминале последовательно отображаются этапы взаимодействия клиента с DHCP-сервером: отправка широковещательного запроса DHCPDISCOVER, получение предложения DHCPOFFER, подтверждающий запрос DHCPREQUEST и финальное подтверждение DHCPACK от сервера с адреса 192.168.0.1. В результате клиенту был назначен IP-адрес 192.168.0.89 с указанием времени аренды.

```
hep2014@hep2014-Raider-GE67HX-12UHS:~$ sudo dhclient -v wlo1
Internet Systems Consortium DHCP Client 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/wlo1/88:d8:2e:f8:7b:5e
Sending on   LPF/wlo1/88:d8:2e:f8:7b:5e
Sending on   Socket/fallback
xid: warning: no netdev with useable HWADDR found for seed's uniqueness enforcement
xid: rand init seed (0x693df89b) built using gethostid
DHCPDISCOVER on wlo1 to 255.255.255.255 port 67 interval 3 (xid=0x832beb1b)
DHCPOFFER of 192.168.0.89 from 192.168.0.1
DHCPREQUEST for 192.168.0.89 on wlo1 to 255.255.255.255 port 67 (xid=0x1beb2b83)
DHCPACK of 192.168.0.89 from 192.168.0.1 (xid=0x832beb1b)
Error: ipv4: Address already assigned.
Setting LLNMR support level "yes" for "3", but the global support level is "no".
bound to 192.168.0.89 -- renewal in 40810 seconds.
```

Рисунок 32 – Работа DHCP-клиента

Параллельно с выполнением команды dhclient в Wireshark был включён захват трафика с фильтром dhcp || bootp. На рисунке 33 показан DHCP-пакет типа Discover. В заголовках видно, что IP-адрес источника равен 0.0.0.0, а адрес назначения - 255.255.255.255, что указывает на широковещательный характер запроса. В поле Client MAC address содержится аппаратный адрес сетевого интерфейса клиента, а в опциях DHCP указано, что тип сообщения — это Discover. Это означает, что клиент ещё не имеет IP-адреса и запрашивает параметры сети у всех доступных DHCP-серверов в сегменте.

```
> Frame 1023: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface wlo1, id 0
> Ethernet II, Src: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 68, Dst Port: 67
> Dynamic Host Configuration Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x832beb1b
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
  Client hardware address padding: 000000000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Discover)
  > Option: (12) Host Name
  > Option: (55) Parameter Request List
  > Option: (255) End
  Padding: 00000000000000000000000000000000
```

Рисунок 33 – DHCP Discover

Следующим этапом является ответ DHCP-сервера с предложением конфигурации. На рисунке 34 представлен DHCP-пакет типа Offer, отправленный сервером с IP-адреса 192.168.0.1 клиенту. В поле Your (client) IP address указан предлагаемый адрес 192.168.0.89. Также в опциях пакета содержатся дополнительные параметры: маска подсети, адрес маршрутизатора по умолчанию, DNS-сервер и значения времени аренды. Данный пакет сообщает клиенту, какие параметры сети могут быть ему выделены.

```

> Frame 1024: 346 bytes on wire (2768 bits), 346 bytes captured (2768 bits) on interface wlo1, id 0
> Ethernet II, Src: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84), Dst: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
> Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.89
> User Datagram Protocol, Src Port: 67, Dst Port: 68
- Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x832beb1b
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 192.168.0.89
  Next server IP address: 192.168.0.1
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Offer)
  > Option: (54) DHCP Server Identifier (192.168.0.1)
  > Option: (51) IP Address Lease Time
  > Option: (58) Renewal Time Value
  > Option: (59) Rebinding Time Value
  > Option: (1) Subnet Mask (255.255.255.0)
  > Option: (28) Broadcast Address (192.168.0.255)
  > Option: (6) Domain Name Server
  > Option: (3) Router
  > Option: (15) Domain Name
  > Option: (255) End

```

Рисунок 34 – DHCP Offer

После получения предложения клиент отправляет подтверждающий запрос. На рисунке 35 показан DHCP-пакет типа Request, в котором клиент указывает запрашиваемый IP-адрес 192.168.0.89 и идентификатор DHCP-сервера 192.168.0.1. Как и на этапе Discover, данный запрос передаётся широковещательно, чтобы все серверы в сегменте знали, какое предложение было принято клиентом.

```

> Frame 1025: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface wlo1, id 0
> Ethernet II, Src: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 68, Dst Port: 67
- Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x832beb1b
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Request)
  > Option: (54) DHCP Server Identifier (192.168.0.1)
  > Option: (50) Requested IP Address (192.168.0.89)
  > Option: (12) Host Name
  > Option: (55) Parameter Request List
  > Option: (255) End

```

Рисунок 35 – DHCP Request

Финальным этапом является подтверждение аренды со стороны сервера. На рисунке 36 представлен DHCP-пакет типа ACK. В нём сервер окончательно подтверждает выдачу IP-адреса 192.168.0.89 клиенту и передаёт полный набор параметров сети: маску подсети 255.255.255.0, адрес широковещания, адрес маршрутизатора, DNS-сервер и время аренды. После получения данного пакета операционная система применяет полученные настройки и начинает использовать сеть в штатном режиме.

```
▶ Frame 3: 375 bytes on wire (3000 bits), 375 bytes captured (3000 bits) on interface wlo1, id 0
▶ Ethernet II, Src: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84), Dst: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
▶ Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.89
▶ User Datagram Protocol, Src Port: 67, Dst Port: 68
▼ Dynamic Host Configuration Protocol (ACK)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x897f06d6
  Seconds elapsed: 1
  ▶ Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 192.168.0.89
  Next server IP address: 192.168.0.1
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  ▶ Option: (53) DHCP Message Type (ACK)
  ▶ Option: (54) DHCP Server Identifier (192.168.0.1)
  ▶ Option: (51) IP Address Lease Time
  ▶ Option: (58) Renewal Time Value
  ▶ Option: (59) Rebinding Time Value
  ▶ Option: (1) Subnet Mask (255.255.255.0)
  ▶ Option: (28) Broadcast Address (192.168.0.255)
  ▶ Option: (6) Domain Name Server
  ▶ Option: (12) Host Name
  ▶ Option: (3) Router
  ▶ Option: (15) Domain Name
  ▶ Option: (255) End
```

Рисунок 36 – Wire Ack

Выводы по анализу.

В ходе анализа DHCP-трафика был экспериментально подтверждён полный цикл автоматической конфигурации сетевого интерфейса с использованием протокола DHCP. Захваченные пакеты наглядно демонстрируют последовательность обмена сообщениями Discover–Offer–Request–ACK, а также широковещательный характер начального запроса клиента. Анализ содержимого DHCP-пакетов показал, что сервер передаёт клиенту не только IP-адрес, но и ключевые параметры сети, необходимые для корректной работы узла. Таким образом, на практике подтверждена роль DHCP как базового служебного протокола, обеспечивающего автоматическую настройку сетевых параметров в локальных сетях.

1.9 Анализ трафика мессенджера Telegram

На первом этапе были зафиксированы исходные параметры сетевого подключения исследуемой системы и запущен захват сетевого трафика в программе Wireshark. Захват осуществлялся с беспроводного интерфейса wlan1, через который выполнялось сетевое взаимодействие приложения Telegram Desktop. Для исключения постороннего трафика был применён фильтр по TCP-соединениям, использующим порт 443, характерный для защищённых соединений:

```
tcp.port == 443
```

Данный фильтр позволил выделить сетевые соединения, используемые мессенджером Telegram для обмена данными с удалёнными серверами.

Далее был произведён анализ IP-заголовков отфильтрованных пакетов. Установлено, что взаимодействие осуществляется между локальным IP-адресом хоста 192.168.0.89 и удалёнными IP-адресами 149.154.167.41, 149.154.167.50, 149.154.167.51, относящимися к серверной инфраструктуре Telegram.

На рисунке 37 представлен пример IPv4-пакета, в котором указаны адрес источника и адрес назначения, а также протокол верхнего уровня — TCP.

No.	Time	Source	Destination	Protocol	Length	Info
2572	46.536846570	149.154.167.50	192.168.0.89	TCP	66	443 → 47826 [ACK] Seq=1049793 Ack=30455 Win=198624 Len=0 TSval=1596207978 TSecr=3879560690
2573	46.536846450	149.154.167.250	192.168.0.89	TCP	66	443 → 48154 [FIN, ACK] Seq=18764 Ack=2981 Win=74526 Len=0 TSval=2275319680 TSecr=2645506495
2574	46.536850917	192.168.0.89	149.154.167.255	TCP	66	48154 → 443 [ACK] Seq=2091 Ack=18705 Win=40900 Len=0 TSval=2545506445 TSecr=2275319680
2575	46.795328561	149.154.167.50	192.168.0.89	TCP	155	443 → 47826 PSH, ACK Seq=1949793 Ack=30455 Win=198624 Len=89 TSval=1596208140 TSecr=3879560690 [TCP segment of a reassembled PDU]
2576	46.795329270	192.168.0.89	149.154.167.50	TCP	66	47826 → 443 [ACK] Seq=30455 Ack=1949792 Win=1671040 Len=0 TSval=3879560992 TSecr=1596208140
2578	47.729769152	192.168.0.89	149.154.167.50	TCP	427	47826 → 443 [PSH, ACK] Seq=30455 Ack=1949792 Win=1671040 Len=361 TSval=1596208140 TSecr=1596208140 [TCP segment of a reassembled PDU]
2579	47.773018394	149.154.167.50	192.168.0.89	TCP	66	443 → 47826 [ACK] Seq=1949792 Ack=30816 Win=201088 Len=0 TSval=1596209222 TSecr=3879561926
2580	47.813221526	149.154.167.50	192.168.0.89	TCP	155	443 → 47826 PSH, ACK Seq=1949792 Ack=30816 Win=201088 Len=89 TSval=1596209260 TSecr=3879561926 [TCP segment of a reassembled PDU]
2581	47.813286272	192.168.0.89	149.154.167.50	TCP	66	47826 → 443 [ACK] Seq=30816 Ack=1949801 Win=1671040 Len=0 TSval=3879562610 TSecr=1596209260
2582	50.585275677	149.154.167.50	192.168.0.89	TCP	539	443 → 47826 PSH, ACK Seq=1949801 Ack=30816 Win=201088 Len=473 TSval=1596211989 TSecr=3879562610 [TCP segment of a reassembled PDU]
2583	50.585339427	192.168.0.89	149.154.167.50	TCP	66	47826 → 443 [ACK] Seq=30816 Ack=1958354 Win=1678656 Len=0 TSval=3879564782 TSecr=1596211989
2584	52.070173959	192.168.0.89	149.154.167.50	TCP	411	47826 → 443 [PSH, ACK] Seq=30816 Ack=1958354 Win=1671040 Len=345 TSval=1596211989 TSecr=1596211989 [TCP segment of a reassembled PDU]
2585	52.138801421	149.154.167.50	192.168.0.89	TCP	155	443 → 47826 PSH, ACK Seq=1958354 Ack=31161 Win=203520 Len=89 TSval=1596213590 TSecr=3879566287 [TCP segment of a reassembled PDU]
2586	52.138807298	192.168.0.89	149.154.167.50	TCP	283	47826 → 443 [PSH, ACK] Seq=31161 Ack=1958443 Win=1671040 Len=217 TSval=1596213590 TSecr=1596213590 [TCP segment of a reassembled PDU]
2587	52.218664974	149.154.167.50	192.168.0.89	TCP	155	443 → 47826 PSH, ACK Seq=1958443 Ack=31378 Win=205984 Len=89 TSval=1596213661 TSecr=3879566335 [TCP segment of a reassembled PDU]
2588	52.259362189	192.168.0.89	149.154.167.50	TCP	66	47826 → 443 [ACK] Seq=31378 Ack=1958532 Win=1671040 Len=0 TSval=3879566456 TSecr=1596213661
2589	54.176322030	149.154.167.50	192.168.0.89	TCP	475	443 → 47826 PSH, ACK Seq=1958532 Ack=31378 Win=205984 Len=469 TSval=1596215955 TSecr=3879566456 [TCP segment of a reassembled PDU]
2590	54.176394114	192.168.0.89	149.154.167.50	TCP	66	47826 → 443 [ACK] Seq=31378 Ack=1958941 Win=1678656 Len=0 TSval=3879568373 TSecr=1596215555
2591	54.253017608	192.168.0.89	149.154.167.41	TCP	74	52310 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3335436529 TSecr=0 WS=128
2592	54.25408542	192.168.0.89	149.154.167.50	TCP	74	55466 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3879566452 TSecr=0 WS=128
2593	54.25723541	192.168.0.89	149.154.167.51	TCP	74	45536 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2387400402 TSecr=0 WS=128
2597	54.289448759	149.154.167.41	192.168.0.89	TCP	74	443 → 52310 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=1303051478 TSecr=3335436529 WS=32
2598	54.289476738	192.168.0.89	149.154.167.41	TCP	66	52310 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3335436566 TSecr=1303051439
2599	54.289914854	192.168.0.89	149.154.167.41	SSL	323	Continuation Data
2600	54.295453474	149.154.167.50	192.168.0.89	TCP	74	443 → 55466 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=915378663 TSecr=3879568452 WS=32
2601	54.295481243	192.168.0.89	149.154.167.50	TCP	66	55466 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3879568492 TSecr=915378663
2602	54.296172393	192.168.0.89	149.154.167.50	SSL	395	Continuation Data
2603	54.297937499	149.154.167.51	192.168.0.89	TCP	74	443 → 45536 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=541290070 TSecr=2387400402 WS=32
2604	54.297972990	192.168.0.89	149.154.167.51	TCP	66	45536 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2387400442 TSecr=541290070
2605	54.299223431	192.168.0.89	149.154.167.51	SSL	239	Continuation Data
2615	54.328665225	149.154.167.41	192.168.0.89	SSL	167	Continuation Data
2616	54.328701768	192.168.0.89	149.154.167.41	TCP	66	52310 → 443 [ACK] Seq=250 Ack=102 Win=64256 Len=0 TSval=3335436605 TSecr=1303051478
2617	54.328807289	192.168.0.89	149.154.167.50	TCP	66	55466 → 443 [ACK] Seq=330 Ack=1 Win=64256 Len=0 TSval=3879568825 TSecr=915378663
2618	54.328922945	192.168.0.89	149.154.167.51	TCP	66	45536 → 443 [FIN, ACK] Seq=174 Ack=1 Win=64256 Len=0 TSval=2387400473 TSecr=541290070
2622	54.334676735	149.154.167.50	192.168.0.89	SSL	191	Continuation Data
2623	54.334727348	192.168.0.89	149.154.167.50	TCP	54	45536 → 443 [RST] Seq=330 Win=0 Len=0
2624	54.335177321	149.154.167.51	192.168.0.89	SSL	253	Continuation Data
2625	54.339218553	192.168.0.89	149.154.167.51	TCP	54	45536 → 443 [RST] Seq=174 Win=0 Len=0
2628	54.358976233	192.168.0.89	149.154.167.41	SSL	303	Continuation Data
2637	54.371287234	149.154.167.50	192.168.0.89	TCP	66	443 → 55466 [FIN, ACK] Seq=126 Ack=331 Win=66624 Len=0 TSval=915378739 TSecr=3879568525
2638	54.371799624	192.168.0.89	149.154.167.50	TCP	54	55466 → 443 [RST] Seq=331 Win=0 Len=0
2641	54.373688743	149.154.167.51	192.168.0.89	TCP	66	443 → 45536 [FIN, ACK] Seq=194 Ack=175 Win=66624 Len=0 TSval=541290146 TSecr=2387400473

Рисунок 37 – Список с фильтрами

Следующим этапом был проанализирован транспортный уровень. Telegram использует протокол TCP, что подтверждается наличием стандартной процедуры установления соединения. В захваченном трафике наблюдаются пакеты с флагами SYN, SYN, ACK и ACK, соответствующие этапам трёхстороннего рукопожатия TCP.

В процессе передачи данных активно используются сегменты с флагами PSH, ACK, что указывает на передачу прикладных данных. Также зафиксированы пакеты с флагами

FIN, ACK, свидетельствующие о корректном завершении соединений, и пакеты с флагом RST, которые указывают на принудительный сброс TCP-сессии.

На рисунке 38 показан пример TCP-пакета с установленным флагом RST, отправленного с локального хоста на сервер Telegram по порту 443.

Наличие таких пакетов свидетельствует о том, что приложение динамически управляет сетевыми соединениями, завершая неактуальные TCP-сессии и иницилируя новые подключения к серверам.

После анализа транспортного уровня был выполнен анализ прикладного уровня. В захваченном трафике полезная нагрузка представлена в виде пакетов типа SSL / TLS Continuation Data. Это означает, что передача данных между клиентом и серверами Telegram осуществляется по защищённому каналу с использованием криптографического протокола TLS.

```

▶ Frame 2625: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface wlo1, id 0
▼ Ethernet II, Src: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e), Dst: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84)
  ▶ Destination: c4:e9:1a:76:06:84 (c4:e9:1a:76:06:84)
  ▶ Source: Intel_f8:7b:5e (88:d8:2e:f8:7b:5e)
  Type: IPv4 (0x0800)
▼ Internet Protocol Version 4, Src: 192.168.0.89, Dst: 149.154.167.51
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 40
  Identification: 0x0000 (0)
  ▶ 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: TCP (6)
  Header Checksum: 0x3d01 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.0.89
  Destination Address: 149.154.167.51
▼ Transmission Control Protocol, Src Port: 45536, Dst Port: 443, Seq: 174, Len: 0
  Source Port: 45536
  Destination Port: 443
  [Stream index: 29]
  ▶ [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 174 (relative sequence number)
  Sequence Number (raw): 764928066
  [Next Sequence Number: 174 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x004 (RST)
  Window: 0
  [Calculated window size: 0]
  [Window size scaling factor: 128]
  Checksum: 0xf09b [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
```

Рисунок 38 – Пример пакета

На рисунке 39 представлен пример TCP-пакета, содержащего зашифрованные данные TLS-сеанса.

2600	54.295453474	149.154.167.50	192.168.0.89	TCP	74 443 - 85466 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=915378663 TSecr=3879568452 WS=32
2601	54.295481243	192.168.0.89	149.154.167.50	TCP	66 55466 - 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3879568492 TSecr=915378663
2602	54.296727301	192.168.0.89	149.154.167.50	SSL	395 Continuation Data
2603	54.297932489	149.154.167.51	192.168.0.89	TCP	74 443 - 45536 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=541290879 TSecr=2387480482 WS=32
2604	54.297972006	192.168.0.89	149.154.167.51	TCP	66 45536 - 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2387480442 TSecr=541290879
2605	54.299223431	192.168.0.89	149.154.167.51	SSL	239 Continuation Data
2606	54.303990292	149.154.167.41	192.168.0.89	TCP	74 80 - 49548 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=3872494500 TSecr=3335436537 WS=32
2607	54.303121726	192.168.0.89	149.154.167.41	TCP	66 49548 - 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3335436579 TSecr=3872494500
2608	54.383418850	192.168.0.89	149.154.167.41	HTTP	499 POST /api HTTP/1.1 (application/x-www-form-urlencoded)
2609	54.313841221	149.154.167.51	192.168.0.89	TCP	74 80 - 42482 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=541290888 TSecr=2387480482 WS=32
2610	54.313854486	192.168.0.89	149.154.167.51	TCP	66 42482 - 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2387480457 TSecr=541290888
2611	54.313119845	149.154.167.50	192.168.0.89	TCP	74 80 - 55138 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1240 SACK_PERM TSval=2868463582 TSecr=3879568469 WS=32
2612	54.313132362	192.168.0.89	149.154.167.50	TCP	66 55138 - 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3879568599 TSecr=2868463582
2613	54.313485588	192.168.0.89	149.154.167.51	HTTP	378 POST /api HTTP/1.1 (application/x-www-form-urlencoded)
2614	54.313416205	192.168.0.89	149.154.167.50	HTTP	535 POST /api HTTP/1.1 (application/x-www-form-urlencoded)
2615	54.328665225	149.154.167.41	192.168.0.89	SSL	167 Continuation Data
2616	54.328761768	192.168.0.89	149.154.167.41	TCP	66 52310 - 443 [ACK] Seq=258 Ack=192 Win=64256 Len=0 TSval=3335436605 TSecr=1389651478
2617	54.328961269	192.168.0.89	149.154.167.50	TCP	66 35466 - 443 [FIN, ACK] Seq=330 Ack=1 Win=64256 Len=0 TSval=3879568525 TSecr=915378663
2618	54.329922045	192.168.0.89	149.154.167.51	TCP	66 45536 - 443 [FIN, ACK] Seq=174 Ack=1 Win=64256 Len=0 TSval=2387480473 TSecr=541290879
2619	54.329128735	192.168.0.89	149.154.167.41	TCP	66 49548 - 80 [FIN, ACK] Seq=434 Ack=1 Win=64256 Len=0 TSval=3335436605 TSecr=3872494500
2620	54.329142289	192.168.0.89	149.154.167.51	TCP	66 42482 - 80 [FIN, ACK] Seq=313 Ack=1 Win=64256 Len=0 TSval=2387480473 TSecr=541290888
2621	54.329222644	192.168.0.89	149.154.167.50	TCP	66 55138 - 80 [FIN, ACK] Seq=478 Ack=1 Win=64256 Len=0 TSval=3879568526 TSecr=2868463582
2622	54.334676735	149.154.167.50	192.168.0.89	SSL	191 Continuation Data
2623	54.334722803	192.168.0.89	149.154.167.50	TCP	54 55466 - 443 [RST] Seq=339 Win=0 Len=0
2624	54.339171521	149.154.167.51	192.168.0.89	SSL	258 Continuation Data
2625	54.339165593	192.168.0.89	149.154.167.51	TCP	54 45536 - 443 [RST] Seq=174 Win=0 Len=0
2626	54.348899718	149.154.167.41	192.168.0.89	HTTP	481 HTTP/1.1 200 OK
2627	54.348909003	192.168.0.89	149.154.167.41	TCP	54 45536 - 80 [RST] Seq=434 Win=0 Len=0
2628	54.359976233	192.168.0.89	149.154.167.41	SSL	363 Continuation Data
2629	54.368745552	149.154.167.51	192.168.0.89	HTTP	381 HTTP/1.1 200 OK
2630	54.369722247	149.154.167.50	192.168.0.89	HTTP	373 HTTP/1.1 200 OK
2631	54.368776344	192.168.0.89	149.154.167.51	TCP	54 42482 - 80 [RST] Seq=313 Win=0 Len=0
2632	54.368789457	192.168.0.89	149.154.167.50	TCP	54 55138 - 80 [RST] Seq=478 Win=0 Len=0
2633	54.371170172	149.154.167.51	192.168.0.89	TCP	66 80 - 42482 [FIN, ACK] Seq=316 Ack=314 Win=66624 Len=0 TSval=541290146 TSecr=2387480473
2634	54.371159201	192.168.0.89	149.154.167.51	TCP	54 42482 - 80 [RST] Seq=478 Win=0 Len=0
2635	54.371192014	149.154.167.50	192.168.0.89	TCP	66 80 - 55138 [FIN, ACK] Seq=308 Ack=471 Win=66624 Len=0 TSval=2868463641 TSecr=3879568526
2636	54.371227241	192.168.0.89	149.154.167.50	TCP	54 55138 - 80 [RST] Seq=471 Win=0 Len=0
2637	54.371231211	149.154.167.41	192.168.0.89	TCP	54 49548 - 80 [FIN, ACK] Seq=159 Ack=331 Win=66624 Len=0 TSval=915378739 TSecr=3879568525
2638	54.371209624	192.168.0.89	149.154.167.50	TCP	54 55466 - 443 [RST] Seq=331 Win=0 Len=0
2639	54.373685374	149.154.167.41	192.168.0.89	TCP	66 80 - 49548 [FIN, ACK] Seq=336 Ack=435 Win=66624 Len=0 TSval=3872494631 TSecr=3335436605
2640	54.373692653	192.168.0.89	149.154.167.41	TCP	54 49548 - 80 [RST] Seq=435 Win=0 Len=0
2641	54.373699743	149.154.167.51	192.168.0.89	TCP	66 443 - 45536 [FIN, ACK] Seq=194 Ack=175 Win=66624 Len=0 TSval=541290146 TSecr=2387480473

Рисунок 39 – Пример пакета

Содержимое передаваемых данных не поддаётся анализу, так как оно зашифровано. В трафике отсутствуют читаемые HTTP-заголовки или открытые текстовые данные, что подтверждает использование защищённого соединения.

Выводы по анализу.

В ходе анализа сетевого трафика мессенджера Telegram установлено, что приложение использует TCP-соединения по порту 443 для взаимодействия с серверами. Передача данных осуществляется в зашифрованном виде с применением протокола TLS, что обеспечивает конфиденциальность передаваемой информации.

Анализ TCP-флагов показал, что Telegram корректно реализует механизмы установления и завершения соединений, а также использует принудительный сброс TCP-сессий при необходимости. Полученные результаты подтверждают, что мессенджер Telegram соответствует современным требованиям сетевой безопасности и не передаёт пользовательские данные в открытом виде.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы был выполнен захват и детальный анализ реального сетевого трафика с использованием утилиты Wireshark, исследованы протоколы ARP, DNS, ICMP, HTTP, FTP и DHCP, а также работа диагностических утилит ping и traceroute. Это позволило на практике изучить структуру протокольных блоков данных, механизмы инкапсуляции и фрагментации пакетов, а также принципы взаимодействия уровней модели OSI при передаче данных в компьютерных сетях.